

Introduction to ML

Marini Mattia

3° semestre 3° anno

Indice

1	Concetti base	4
1.1	Tipi di errori	6
2	KNN	7
2.1	Tipi di distanza	7
2.1.1	Distanza euclidea	7
2.1.2	Distanza di minkoski	8
2.1.3	Cosine similarity	8
2.2	Normalizzazione	8
2.3	Costo computazionale	8
3	Perceptron	9
3.1	Multi class classification	10
3.2	Ova	11
3.3	AVA	12
3.4	Calcolo precisione	13
3.5	Decision trees	13
3.6	Training	14
3.6.1	Accuracy gain	14
3.6.2	Information gain	14
3.6.3	Gini index	15
3.6.4	Bootstrapping	15
4	Gradient descent	16
4.1	Batch/stochastic gradient descent (BGD vs SGD)	17
4.2	Gradieent descent con momentum	18
5	SVN	18
5.1	Slack variables	19
5.2	Tipi problemi di programmazione	20
5.2.1	Programmazione lineare	20
5.2.2	Programmazione quadratica	20
5.2.3	Programmazione non lineare	20
5.2.4	Problema duale - caso separabile	20
5.2.5	Problema duale - caso non separabile	21

5.2.6	Kernel trick	21
5.3	Regularizzazione	22
6	Unsupervised learning - riduzione dimensione	23
6.0.1	Media e varianza	23
6.0.2	Autovalori e autovettori	24
6.0.3	Obiettivo	26
6.0.4	Equivalenza metodi:	27
6.0.5	Pseudocodice	28
6.0.6	Trovare di quanto ridurre	29
7	Clustering	29
7.1	k-means	29
7.1.1	Pseudocodice	29
7.1.2	Note	30
7.2	EM-clustering	30
7.3	Spectral clustering	31
7.4	Clustering gerarchico	32
7.4.1	Pseudocodice	32
7.4.2	Vantaggi rispetto a k-means	33
8	Reti neurali	33
8.1	Calcolo output predetto	34
8.1.1	Classificazione: softmax + cross-entropy	34
8.1.2	Regressione: softmax + cross-entropy	35
8.2	Backpropagation	35
8.3	Reti neurali con layer specializzati	37
8.3.1	Convoluzione	37
8.3.2	Operazioni non lineari	37
8.3.3	Operazioni di spatial pooling	38
8.4	Altri reti neurali	38
8.4.1	Reti neurali ricorrenti (RNN)	38
8.4.2	Autoencoders	39
9	Modelli generativi	39
9.1	Density estimation	39
9.2	VAE	40
9.2.1	Conditional VAE	42
9.2.2	Problemi	42
9.3	GAN	42
9.3.1	Problemi	43
10	Diffusion models	43
10.1	Training	45
10.2	U-net	45
10.3	Altre tecniche avanzate	46
10.3.1	Diffusion GANs	46
10.3.2	Distillazione	46
10.3.3	Latent space diffusion models	47

10.3.4 Text to image diffusion models	47
11 Reinforcement learning	48
11.1 Markov Decision Process (MDP)	48
11.2 Metodo basato sul valore	49
12 Miscellanea nozioni	52
12.1 Cosine distance	52

Tipi di learning: in base ai dati che vengono dati in input all'algoritmo di machine learning si possono sfruttare più strategie di apprendimento.

Definizione 1: *Metodi di learning*

Dati un insieme di dati $\mathcal{D}_n$ detto *training set*, dove X è l'insieme contenente tutte le possibili features e Y è l'insieme di tutte le possibili labels, allora abbiamo i seguenti tipi di learning:

- Supervised learning: vengono forniti dati etichettati, ossia ogni dato di input x_i ha una label y_i associata

$$\mathcal{D}_n = \{(x_1, y_1), \dots, (x_n, y_n)\}$$

- Unsupervised learning: vengono forniti dati non etichettati:

$$\mathcal{D}_n = \{(x_1), \dots, (x_n)\}$$

- Reinforced learning: vengono forniti dati non etichettati, ma, interagendo con l'ambiente, l'algoritmo riceve dei reward o delle penalizzazioni, permettendo l'apprendimento. Ad esempio il punteggio in un videogioco
- Semi-supervised learning: vengono forniti dati parzialmente etichettati, solitamente la maggiorparte viene data *senza etichetta*. Il modello sfrutta la piccola quantità di dati etichettati per imparare a fare delle previsioni, e poi usa i dati non etichettati per migliorare la propria generalizzazione, spesso tramite tecniche di regolarizzazione, clustering o inferenza delle label.
- Active learning: il modello può selezionare in modo attivo i dati dai quali desidera imparare (cioè chiedere all'utente di etichettare dati "strategici"). Questo in quanto l'etichettatura dei dati manuale può essere molto costosa (es. immagini mediche)

Il processo di learning può avvenire *online*, o *offline*. Nel primo caso il modello è in continua evoluzione una volta deployato, nel secondo caso lo stato del modello è congelato al momento del deployment, ossia impara solo durante la fase di training, tutto d'un colpo

Definizione 2: Task e spazio ipotesi

Una task è l'insieme di funzioni che sono adeguate a risolvere un determinato problema (ad esempio classificare frutti). Dati un insieme di dati X e un insieme di labels Y , allora una task è un insieme di funzioni:

$$f : X \rightarrow Y \quad f_{\text{task}} \in Y^X$$

dove Y^X è l'insieme di tutte le funzioni da X a Y :

$$Y^X = \{f : X \rightarrow Y\}$$

delle funzioni in f_{task} mi limito ad analizzare un subset di interesse, detto *spazio di ipotesi*:

$$H \subset f_{\text{task}}$$

In base al tipo di task, abbiamo diversi obiettivi:

Classificazione

Trova una funzione $f \in Y^X$ che assegna ad ogni $x \in X$ un label $y \in Y$

Regressione

Trova una funzione f tale per cui $f(x) \in Y$, che associ ad un input $x \in X$ un valore reale

Clustering

Trova una funzione $f \in \mathbb{N}^X$ che assegna ad ogni $x \in X$ un valore discreto $\in \mathbb{N}$, ossia un indice del cluster

Dimensionality reduction

Trova una funzione $f \in Y^X$ che mappa ogni valore $x \in X$ ad un valore $y \in Y$ dove $\dim(Y) \ll \dim(X)$

Density estimation

Trova una funzione $f \in \Delta(X)$ approssimi meglio la distribuzione dei dati in X . $\Delta(X)$ rappresenta tutte le possibili distribuzioni di probabilità definite su X

Definizione 3: Dati (p_{data})

I dati su di cui lavora seguono una distribuzione identificata da p_{data} .

- o Classification and Regression:

$$p_{\text{data}} \in \Delta(X \times Y)$$

- o Density estimation, Clustering and Dimensionality Reduction

$$p_{\text{data}} \in \Delta(X)$$

L'obiettivo è quello di ridurre l'errore complessivo su tutti i possibili input dei dati su cui opererà l'algoritmo. Abbiamo più definizioni di obiettivo:

Definizione 4: Obiettivo (target)

Idealmente vorremmo minimizzare l'errore in generale, ma questo non è possibile in quanto siamo costretti a limitarci ad un insieme di funzioni $\mathcal{H} \in \mathcal{F}_{\text{task}}$ e non conosciamo la data generating distribution p_{data} , quindi dobbiamo operare sul training set $\mathcal{D}_n$

- Ideal target:

$$f^* \in \arg \min_{f \in \mathcal{F}_{\text{task}}} E(f; p_{\text{data}})$$

- Feasible target

$$f_{\mathcal{H}}^* \in \arg \min_{f \in \mathcal{H}} E(f; p_{\text{data}})$$

- Actual target

$$f_{\mathcal{H}}^*(\mathcal{D}_n) \in \arg \min_{f \in \mathcal{H}} E(f; \mathcal{D}_n)$$

L'errore complessivamente è misurato come somma di tutti gli errori su qualsiasi dato $\in \mathcal{D}_n$:

Definizione 5: Error

L'errore complessivamente è misurato come somma di tutti gli errori su qualsiasi dato $\in \mathcal{D}_n$:

$$E(f; p_{\text{data}}) = \mathbb{E}_{z \sim p_{\text{data}}} [\ell(f; z)]$$

$$E(f; \mathcal{D}_n) = \frac{1}{n} \sum_{i=1}^n \ell(f; z_i)$$

L'errore è definito spesso tramite la poinwise loss, di fatto quanto di discosta la previsione dal'etichetta effettiva

1.1 Tipi di errori

- *Underfitting*: l'algoritmo non si specifica abbastanza sui dati di training, non riuscendo a catturare i pattern sottostanti. Grande errore sul training set
- *Overfitting*: l'algoritmo si specializza *troppo*, finendo per generalizzare male sui dati non visti. Piccolo errore sul training set, grande errore sui dati non visti

Inoltre, possiamo parlare di

- *Errore di approssimazione*: errore dovuto al fatto che non consideriamo tutte le funzione $f \in \mathcal{F}_{\text{task}}$ ma solo quelle in $\mathcal{H}$
- *Errore irriducibile*: errore che non può essere evitato per via di fattori che riguardano la naturale variabilità di un sistema

Definizione 6: *Lazy/eager learning*

- *Eager learning*: l'algoritmo costruisce un modello generale basato sui dati di training, per poi usarlo per fare previsioni su dati non visti (es decision trees)
- *Lazy learning*: l'algoritmo memorizza i dati di training e rimanda la generalizzazione fino al momento in cui deve fare una previsione su un dato non visto (es KNN)

2

KNN

Il k-nearest-neighbor è una tecnica utilizzata per il supervised learning che si basa sulla rappresentazione delle features in spazi n -dimensionali. Intuitivamente, possiamo associare un valore numerico ad ogni $x_1, \dots, x_n$ appartenente alle tuple delle features, in maniera tale che ogni $x \in \mathcal{D}_m$ sia una coordinata in uno spazio n -dimensionale.

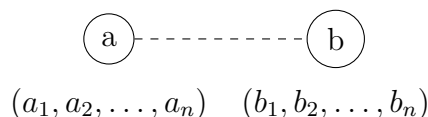
Dato un dato appartenente al test set, vengono considerati gli n punti più vicini e viene assegnata la label più presente in questi k punti al nuovo dato. Il criterio di distanza può essere dato, ad esempio, dalla distanza euclidea

2.1 Tipi di distanza

I criteri di distanza o di quanto sono simili i dati sono spesso 2:

- Similarità
 - Misura numerica di quanto siano simili due oggetti di dati.
 - È più alta quando gli oggetti sono più simili.
 - Spesso rientra nell'intervallo $[0,1]$
- Distanza
 - Misura numerica di quanto siano diversi due oggetti di dati.
 - È più bassa quando gli oggetti sono più simili.
 - La dissimilarità minima è spesso 0.
 - Il limite superiore varia.

2.1.1 Distanza euclidea


$$(a_1, a_2, \dots, a_n) \quad (b_1, b_2, \dots, b_n)$$

$$D(a, b) = \sqrt{(a_1 - b_1)^2 + (a_2 - b_2)^2 + \dots + (a_n - b_n)^2}$$

2.1.2 Distanza di minkoski

La distanza di Minkowski è una generalizzazione della distanza euclidea

$$D(a, b) = \left(\sum_{k=1}^p |a_k - b_k|^r \right)^{1/r}$$

r è un parametro, p è il numero di dimensioni

- $r = 1$. Distanza a blocchi di città (Manhattan, norma L1)
- $r = 2$. Distanza euclidea
- $r \rightarrow \infty$. Distanza "supremo" (norma L_∞). Questa è la differenza massima tra qualsiasi componente dei vettori

2.1.3 Cosine similarity

Questo è un indice di quanto i vettori siano allineati nello spazio. Se i vettori hanno stessa direzione e stesso verso vale 1, se sono ortogonali vale 0, se hanno stessa direzione e verso opposto vale -1. Generalizza bene a spazi n -dimensionali

$$\cos(d_1, d_2) = \frac{d_1 \cdot d_2}{\|d_1\| \|d_2\|}$$

Dove $\cdot$ indica il prodotto scalare tra vettori e $\|d\|$ è la lunghezza (norma) del vettore d .

2.2 Normalizzazione

Se i dati sono di scala diversa è utile applicare la standardizzazione affinché ogni feature abbia la stessa importanza nel calcolo della distanza:

- **Standardization or Z-score normalization** : Rescale the data so that the mean is 0 and the standard deviation from the mean (standard scores) is 1

$$x_{\text{norm}} = \frac{x - \mu}{\sigma}$$

μ is mean, σ is a standard deviation from the mean (standard score)

- **Min-Max scaling** : Scale the data to a fixed range – between 0 and 1

$$x_{\text{norm}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

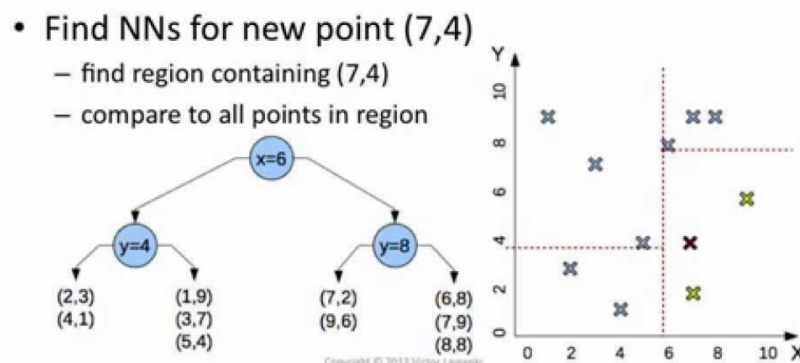
2.3 Costo computazionale

Avendo $|\mathcal{D}_n| = n$ e ciascun dato con m features, allora la complessità per classificare un campione è di:

$$\Theta(n \cdot m)$$

Il costo può essere migliorato costruendo un $K - d$ tree. Il processo è il seguente:

- Prendo una dimensione d e divido i dati in due metà in base alla mediana di quella dimensione (ordino i dati in maniera crescente e la prima metà la assegno ad un insieme, la seconda ad un altro)
- Ripeto ricorsivamente la stessa procedura sui due insiemi ottenuti, dividendo sulla base della seconda dimensione
- Fra i punti rimanenti calcolo la distanza come nell'algoritmo KNN classico



Il problema di questo algoritmo è il cosiddetto *curse of dimensionality*. In particolare, con l'aumentare delle features, dunque delle dimensioni, i punti tendono ad essere molto distanti tra di loro. Per mantenere l'efficacia, è dunque necessario avere dataset molto densi, in particolare che crescono esponenzialmente nel numero di features n . Questo è un problema computazionale e di data availability

3 Perceptron

Definizione 7: *Bias*

Il bias di un modello indica quanto forti sono le assunzioni che si fanno sulla distribuzione dei dati.

- I classificatori a basso bias fanno poche assunzioni sui dati (k-NN e gli alberi decisionali sono generalmente considerati a basso bias)
- I classificatori ad alto bias fanno forti assunzioni sui dati

Il perceptron non fa altro che assumere che i dati siano linearmente separabili, ossia che esista un iperpiano che separa le due classi. In particolare, il perceptron cerca di trovare i pesi $w_1, \dots, w_n$ e il bias b tali per cui dati sono separati dall'iperpiano:

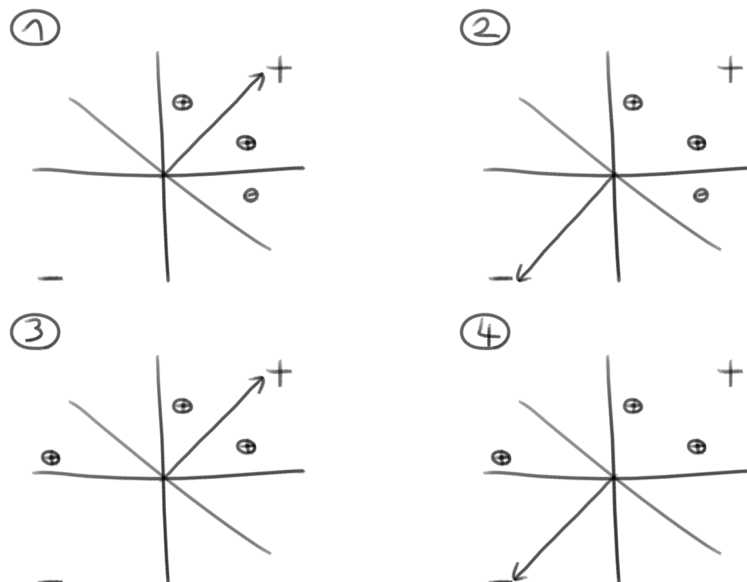
$$0 = f_1 w_1 + f_2 w_2 + \dots + f_n w_n + b = b + \sum_{i=1}^n f_i w_i$$

Algoritmo 1: *Perceptron learning algorithm*

```
while not converged (or for some number of iterations) do  
  foreach training example  $(f_1, f_2, \dots, f_n, \text{label})$  do  
     $\text{prediction} \leftarrow b + \sum_{i=1}^n w_i f_i$ ;  
    if prediction is different from label then  
      for each  $w_i$  do  
         $w_i \leftarrow w_i + f_i \times \text{label}$ ;  
       $b \leftarrow b + \text{label}$ ;
```

Algorithm 1: Perceptron learning algorithm

Nota che in realtà non è detto che ad ogni step la soluzione migliori, ma è garantito che se i dati sono linearmente separabili allora la soluzione converge. Prendi questa immagine e considera i casi 2 e 4: qui la soluzione ruota il piano nella direzione sbagliata!



3.1 Multi class classification

Definizione 8: *Binary classifier*

Un *binary classifier* è un classificatore che assegna ad ogni dato una delle due possibili classi, solitamente $\mathcal{Y} = \{+1, -1\}$

Formalmente dato:

1. Uno spazio di input $\mathcal{X}$
2. Una distribuzione sconosciuta $\mathcal{D}$ su $\mathcal{X} \times \{-1, +1\}$
3. Un insieme di addestramento D campionato da $\mathcal{D}$

La *classificazione binaria* ha come scopo quello di trovare una funzione f che minimizza:

$$\mathbb{E}_{(x,y) \sim \mathcal{D}} [f(x) \neq y]$$

La classificazione multiclassi invece ha il set di label contenente k classi:

Definizione 9: *Multiclass classification*

Un *multiclass classifier* è un classificatore che assegna ad ogni dato una delle k possibili classi ($|\mathcal{Y}| = k$)

Formalmente dato:

1. Uno spazio di input $\mathcal{X}$ e il numero di classi K
2. Una distribuzione sconosciuta $\mathcal{D}$ su $\mathcal{X} \times [K]$
3. Un insieme di addestramento D campionato da $\mathcal{D}$

La *classificazione multiclasse* ha come scopo quello di trovare una funzione f che minimizza:

$$\mathbb{E}_{(x,y) \sim \mathcal{D}} [f(x) \neq y]$$

Con KNN possiamo ottenere classificazione multiclasse senza modifiche all'algoritmo, mentre con il perceptron dobbiamo usare delle tecniche, più in particolare dobbiamo reiterare la separazione binaria per ogni classe.

3.2 Ova

L'idea dell'OVA è di applicare k classificazioni binarie. Chiamando k_i l' i -esima classe, allora alleno un modello di classificazione binaria distinguendo tra $x \in k_i$ e $x \notin k_i$. C'è tuttavia il problema della confidence. Lo spazio viene infatti diviso in parti linearmente, ma alcune di queste non identificano univocamente un campione, ad esempio: dunque si definisce lo score come la distanza dall'iperpiano:

$$\text{score}_i = b_i + \sum_{j=1}^k w_{ij} x_{ij}$$

Il maggiore di questi indica il fatto che il punto x_i appartiene maggiormente alla classe k_i . Se il punto non appartiene a nessuna classe, quindi tutti gli score sono negativi, allora

viene selezionato ancora lo score più grande(meno negativo), ossia la classe al quale il punto è più vicino

Algoritmo 2: *Training OVA*

Input: $D^{multiclass}$, BINARYTRAIN
Output: $f_1, \dots, f_K$

for $i = 1$ **to** K **do**
 $D^{bin} \leftarrow$ riclassifica $D^{multiclass}$ così che la classe i sia positiva e $\neg i$ sia negativa;
 $f_i \leftarrow$ BINARYTRAIN(D^{bin});
return $f_1, \dots, f_K$

Algoritmo 3: *Classificazione OVA*

Input: $f_1, \dots, f_K, \hat{x}$
Output: classe con il punteggio massimo

$score \leftarrow \langle 0, 0, \dots, 0 \rangle$; ▷ inizializza K punteggi a zero
for $i = 1$ **to** K **do**
 $y \leftarrow f_i(\hat{x})$;
 $score_i \leftarrow score_i + y$;
return $\arg \max_k score_k$

3.3 AVA

L'idea dell'AVA (*All-Versus-All*, detto anche *One-vs-One*) è di applicare una classificazione binaria per *ogni* coppia distinta di classi. Se ci sono k classi, si addestrano $\frac{k(k-1)}{2}$ modelli binari, ciascuno incaricato di distinguere tra gli elementi delle sole due classi coinvolte.

Siano k_i e k_j due classi distinte. Per ogni coppia (i, j) con $i < j$, si addestra un classificatore binario f_{ij} che distingue tra $x \in k_i$ (positivo) e $x \in k_j$ (negativo), ignorando tutti i campioni appartenenti ad altre classi.

In fase di classificazione, si incrementa lo score della classe i se testando tramite il modello binario f_{ij} il campione appartiene a k_i , e viceversa. Formalmente, dato un campione $\hat{x}$ e supponendo che $f_{ij}(\hat{x}) > 0$ se $\hat{x}$ ha label in k_j

$$\begin{aligned} score_j &+ = f_{ij}(\hat{x}) \\ score_i &+ = f_{ij}(\hat{x}) \end{aligned}$$

In alternativa, si può semplicemente aggiungere 1 allo score, anziché considerare la distanza dall'iperpiano. Questa viene detta versione *unweighted*

Algoritmo 4: *Training AVA*

Input: $D^{\text{multiclasse}}$, BINARYTRAIN
Output: $f_{ij} \quad \forall 1 \leq i < j \leq k$

$f_{ij} \leftarrow \emptyset, \forall 1 \leq i < j \leq K;$
for $i = 1$ **to** $K - 1$ **do**
 $D^{\text{pos}} \leftarrow$ tutti $x \in D^{\text{multiclasse}}$ etichettati i ;
 for $j = i + 1$ **to** K **do**
 $D^{\text{neg}} \leftarrow$ tutti $x \in D^{\text{multiclasse}}$ etichettati j ;
 $D^{\text{bin}} \leftarrow \{(x, +1) : x \in D^{\text{pos}}\} \cup \{(x, -1) : x \in D^{\text{neg}}\};$
 $f_{ij} \leftarrow \text{BINARYTRAIN}(D^{\text{bin}});$
return $f_{ij} \quad \forall 1 \leq i < j \leq k$

Algoritmo 5: *Classificazione AVA*

Input: tutti i f_{ij} , $\hat{x}$
Output: classe con il punteggio massimo

$score \leftarrow \langle 0, 0, \dots, 0 \rangle;$ $\triangleright$ inizializza K punteggi a zero
for $i = 1$ **to** $K - 1$ **do**
 for $j = i + 1$ **to** K **do**
 $y \leftarrow f_{ij}(\hat{x});$
 $score_i \leftarrow score_i + y;$
 $score_j \leftarrow score_j - y;$
return $\arg \max_k score_k$

3.4 Calcolo precisione

- *Microaveraging*: media complessiva dei casi classificati correttamente

$$\frac{\text{casi classificati correttamente}}{\text{casi totali}}$$

- *Macroaveraging*: media fra le medie dei casi classificati correttamente all'interno delle singole classi. Date n classi

$$\frac{\sum_{k=1}^n \frac{\text{casi classificati correttamente della classe } k}{\text{campioni appartenenti alla classe } k}}{n}$$

da più importanza alle label rare

3.5 Decision trees

L'idea è quella di dividere ad ogni nodo il dataset in due sottoinsiemi in base al valore di una feature, in modo tale da massimizzare la purezza delle classi nei sottoinsiemi. Quando si è arrivati a una foglia, si decide la categoria da associarvi in base a:

- L'elemento prevalente nel caso della classificazione:

$$y^m \leftarrow \operatorname{argmax}_{t \in \{1, \dots, C\}} \sum_{m_i} \mathbf{1} \{t = t^{(m_i)}\}$$

- La media dei valori nel caso della regressione:

$$y^m \leftarrow \frac{1}{N_m} \sum_{m_i} y^{(m_i)}$$

3.6 Training

Trovare un tree ideale (che sia quindi piccolo e con abbastanza campioni per ogni spazio) è un problema *NP-hard*, quindi si usano euristiche greedy:

- Splitto in base al miglior attributo (seglendolo in base al *gain* della split)
- Ripeto ricorsivamente per i sottoinsiemi ottenuti

Il criterio secondo cui scegliere il miglior attributo può essere basato su:

- Maggiore accuracy gain
- Maggiore information gain
- Minore GINI index

3.6.1 Accuracy gain

Chiamando l'errore per una regione R $L(R)$, allora confrontiamo l'errore prima e dopo che venga splittata la regione nelle regioni R_1 e R_2 :

- Errore prima:

$$L(R)$$

- Errore dopo:

$$\frac{|R_1|}{|R|} L(R_1) + \frac{|R_2|}{|R|} L(R_2)$$

L'*accuracy gain* è la differenza fra i due errori:

$$\text{accuracy gain} = L(R) - \frac{|R_1| L(R_1) + |R_2| L(R_2)}{|R|}$$

3.6.2 Information gain

Uno split che è effettivamente buono può avere un gain basso o nullo. Per questo dobbiamo usare il concetto di *certainty* per compensare questo problema. In particolare:

- Tutti gli esempi in una regione appartengono alla stessa classe (bassa *uncertainty*, bene)
- Tutti gli esempi in una regione sono equamente distribuiti tra le classi (alta *uncertainty*, male)

L'uncertainty si può misurare con l'entropia, che indica quanto è difficile prevedere l'esito di un esperimento casuale.

$$H(X) = -\mathbb{E}_{X \sim p}[\log_2 p(X)] = -\sum_{x \in \mathcal{X}} p(x) \log_2 p(x)$$

in maniera analoga all'accuracy gain si può parlare di *information gain*, ossia di quanto l'entropia diminuisce dopo uno split:

$$IG = H(R) - \frac{|R_1| H(R_1) + |R_2| H(R_2)}{|R|}$$

3.6.3 Gini index

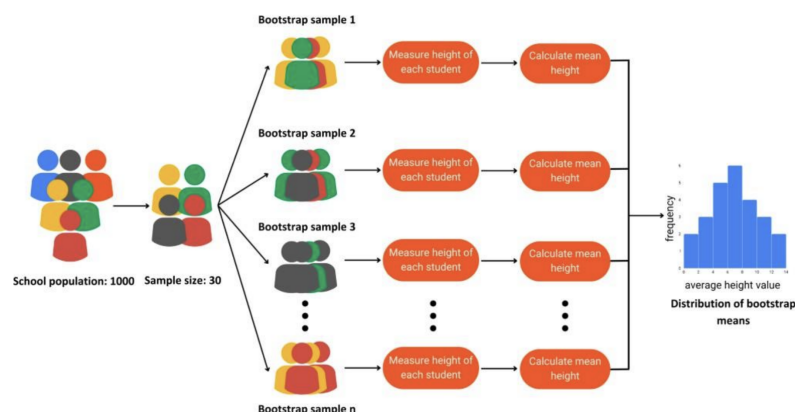
L'indice *GINI* indica quanto una classe è mescolata, un po' in maniera opposta a quanto fa l'entropia. Una classe completamente pura avrà indice GINI pari a 0, mentre

In maniera simile all'entropia vale 0 quando tutti gli esempi appartengono alla stessa classe, e vale massimo quando gli esempi sono equamente distribuiti tra le classi.

$$\text{GINI}(R) = 1 - \sum_j [p(j|R)]^2$$

3.6.4 Bootstrapping

Il resampling è una tecnica che consiste nel creare nuovi dataset a partire da un dataset originale, campionando con o senza rimpiazzo. Si campionano dataset di dimensione inferiore randomicamente dal dataset iniziale e si esegue microaveraging



Definizione 10: *Bagging*

Il *bagging* (bootstrap aggregating) è una tecnica di ensemble learning che utilizza il resampling per migliorare la stabilità e l'accuratezza dei modelli di apprendimento automatico. Si creano più modelli indipendenti su diversi campioni del dataset e si combinano le loro previsioni tramite votazione (per classificazione) o media (per regressione)

Una random forest non è altro che un insieme di decision trees il cui risultato è aggregato come votazione(per classificazione) o come media(per regressione)

4

Gradient descent

Per valutare la qualità di un modello, si usa una funzione di costo che misura la differenza fra i valori previsti e quelli reali. L'obiettivo è minimizzare questa funzione di costo.

Definizione 11: *Funzione di loss*

Una *funzione di loss* è una funzione che misura la differenza tra i valori previsti e quelli reali.

Ad esempio, per un modello di classificazione lineare, la funzione di loss può essere il numero di punti che stanno dalla parte sbagliata del piano

$$\text{loss}(w, b) = \sum_{i=1}^n \mathbf{1}[y_i(w \cdot x_i + b) < 0]$$

l'obiettivo è minimizzare questa funzione ma spesso è molto difficile (NP-hard in fact). Spesso quindi si cerca un'approssimazione, cercando i minimi locali in funzioni che limitano la funzione di loss dall'alto:

Definizione 12: *Funzione di loss surrogata*

Una *funzione di loss surrogata* è una funzione che approssima la funzione di loss, limitandola dall'alto.

Di seguito le seguenti funzioni di loss più comuni:

0-1 loss	$\ell_{0/1}(y, y') = \mathbf{1}[y y' \leq 0]$
hinge loss	$\ell_{\text{hinge}}(y, y') = \max(0, 1 - y y')$
exponential loss	$\ell_{\text{exp}}(y, y') = \exp(-y y')$
square loss	$\ell_{\text{sq}}(y, y') = (y - y')^2$

Per trovare il minimo di una funzione di loss surrogata posso usare il *gradient descent*, cercando di "scivolare" verso il minimo andando "in discesa" verso ogni asse singolarmente. Ad esempio utilizzando la loss function esponenziale

$$\text{loss}(w, b) = \sum_{i=1}^n \exp(y_i(w \cdot x_i + b))$$

in particolare se cerco di modificare un peso alla volta considerando la loss function in una sola delle sue k dimensioni, ho che:

$$\begin{aligned} w_j &= w_j - \eta \frac{d}{dw_j} \text{loss}(w, b) \\ &= w_j - \eta \sum_{i=1}^n -y_i x_{ij} \exp(-y_i(w \cdot x_i + b)) \end{aligned}$$

dunque abbiamo ottenuto la formula per aggiornare ciascun peso:

$$w_j = w_j + \eta \sum_{i=1}^n y_i x_{ij} \exp(-y_i(w \cdot x_i + b))$$

il che equivale a considerare ciascun punto nel training set e aggiornare il peso j -esimo come segue:

$$w_j = w_j + \eta y_i x_{ij} \exp(-y_i(w \cdot x_i + b)) \quad \forall i \in [0, \dots, n]$$

Algoritmo 6: *Gradient descent*

Input: Learning rates $\eta = [\eta_1, \dots, \eta_n]$, loss function L , # of iterations K

Output: Weights $w = [w_1, \dots, w_k]$

gradient_descent(X, Y, L, η):

```

     $w^{(0)} = [0, \dots, 0]$  ;                                ▷ Dati come input in alternativa
    for  $k \in [1, \dots, K]$  do
         $g^{(k)} = \nabla_{w^{(k)}} L|_{w^{(k-1)}}$  ;                    ▷ shift for each direction
         $w^{(k)} = w^{(k-1)} - \eta^{(k)} g^{(k)}$  ;                ▷ move in direction of gradient
    return  $w^{(K)}$ 
```

Nota che $w^{(k)}$ indica i pesi relativi alla k -esima iterazione, ossia dopo essere stati migliorati k volte. Similmente $g^{(k)}$ è il vettore contenente le derivate parziali (ossia considerate una direzione alla volta) della loss function nel punto ottenuto dopo k iterazioni:

$$\nabla_{w^{(k)}} L = \begin{bmatrix} \frac{\partial L}{\partial w_1^{(k)}} & \frac{\partial L}{\partial w_2^{(k)}} & \cdots & \frac{\partial L}{\partial w_N^{(k)}} \end{bmatrix}$$

dove L è la *funzione di loss*

4.1 Batch/stochastic gradient descent (BGD vs SGD)

Algoritmo 7: *Batch gradient descent*

Input: Learning rate η_k , initial parameters w

BGD(X, Y, L, w, η):

```

     $w^{(0)} = [0, \dots, 0]$  ;                                ▷ Dati come input in alternativa
    while stopping criteria not met do
        ▷ Calcolo gradiente su  $N$  examples:
         $g \leftarrow \frac{1}{N} \nabla_w \sum_i L(f(x_i; w), y_i)$ 
        ▷ Applico update:
         $w \leftarrow w - \eta_k g$ 
```

Nello *stochastic gradient descent* invece, ad ogni iterazione si considera un solo esempio (o un piccolo batch di esempi) per calcolare il gradiente:

Algoritmo 8: *Stochastic gradient descent*

Input: Learning rate η_k , initial parameters w

SGD(X, Y, L, w, η):

```
while stopping criteria not met do
    ▷ Calcolo gradiente su  $N$  examples:
       $g \leftarrow \nabla_w L(f(x_i; w), y_i)$ 
    ▷ Applico update:
       $w \leftarrow w - \eta_k g$ 
```

Di fatto la differenza sta nel calcolare il gradiente su tutti i dati (BGD) o su un singolo dato (SGD). In questo secondo caso, il gradiente sarà più rumoroso, ma l'algoritmo convergerà più velocemente.

Batch gradient descent

$$g \leftarrow \frac{1}{N} \nabla_w \sum_i L(f(x_i; w), y_i)$$

Stochastic gradient descent

$$g \leftarrow \nabla_w L(f(x_i; w), y_i)$$

4.2 Gradient descent con momentum

Si può simulare una "discesa con inerzia" aggiungendo un termine di momentum al calcolo del gradiente. In questo modo si riesce a superare più facilmente i minimi locali o a scendere più velocemente:

Algoritmo 9: *Stochastic gradient descent con momentum*

Input: Learning rate η_k , initial parameters w , initial velocity v , momentum α

SGD_momentum($X, Y, L, w, \eta, v, \alpha$):

```
while stopping criteria not met do
    ▷ Calcolo gradiente su  $N$  examples:
       $g \leftarrow \nabla_w L(f(x_i; w), y_i)$ 
    ▷ Calcolo velocità:
       $v \leftarrow \alpha v - \eta_k g$ 
    ▷ Applico update:
       $w \leftarrow w + v$ 
```

5 SVN

Possiamo pensare che all'interno di un perceptron sia opportuno cercare un piano tale per cui la distanza fra i vari punti è massima, non uno a caso che li separa. Questo è ciò che fanno le *support vector machines*.

L'idea è di cercare due iperpiani, uno nel senso positivo e uno nel senso negativo, tali per cui la distanza tra i due è massima.

$$wx_i + b = 1 \qquad wx_i + b = -1$$

Si noti che il $+1$ e -1 *non* impongono alcuna distanza da piano centrale, ma servono solo per definire i due piani paralleli. La distanza tra dal piano parallelo è data da:

$$\text{distanza} = \frac{wx_i + b}{\|w\|} = \frac{1}{\|w\|}$$

otteniamo dunque il problema di ottimizzazione quadratica:

$$\begin{aligned} \max_{w,b} \frac{1}{\|w\|} &= \min_{w,b} \|w\| = \min_{w,b} \|w\|^2 \\ y_i(w \cdot x_i + b) &\geq 1 \quad \forall i \end{aligned}$$

5.1 Slack variables

Se i dati non sono linearmente separabili allora sarà impossibile trovare una soluzione al problema. Per questo si introducono le *slack variables* $\xi_i \geq 0$, che permettono ad alcuni punti di essere "spostati" dalla parte corretta del piano. In particolare, con $|\mathcal{D}_n| = n$ si avranno n slack variables.

$$\begin{aligned} \min_{w,b} \|w\|^2 + \mathcal{C} \sum_i \mathcal{S}_i \\ y_i(w \cdot x_i + b) &\geq 1 - \mathcal{S}_i \quad \forall i, \mathcal{S}_i \geq 0 \end{aligned}$$

- Nella prima parte vogliamo tenere conto della penalità data dall'uso delle slack variables
- Nella seconda parte, se uno slack variable $\mathcal{S}_i$ è maggiore di 0, allora il punto x_i non è correttamente classificato
- Il parametro $\mathcal{C}$ bilancia l'importanza della massimizzazione del margine e la minimizzazione dell'errore di classificazione dovuto alle slack variables, utile per controllare over/underfitting

Dato l'iperpiano ideale, possiamo trovare i valori delle slack variables come segue:

$$\mathcal{S}_i = \begin{cases} 0 & \text{if } y_i(w \cdot x_i + b) \geq 1 \\ 1 - y_i(w \cdot x_i + b) & \text{otherwise} \end{cases}$$

che in un one-liner diventa:

$$\mathcal{S}_i = \max(0, 1 - y_i(w \cdot x_i + b))$$

Tenendo conto di questo possiamo rendere il problema di soft-svm un problema unconstrained, dato che non abbiamo più bisogno di assicurarci che i punti siano dalla parte corretta del piano:

$$\begin{aligned} \min_{w,b} \|w\|^2 + \mathcal{C} \sum_i \mathcal{S}_i \\ \min_{w,b} \|w\|^2 + \mathcal{C} \sum_i \max(0, 1 - y_i(w \cdot x_i + b)) \end{aligned}$$

Si noti che $\max(0, 1 - y_i(w \cdot x_i + b)) = \text{loss}_{\text{hinge}}(y_i, y'_i)$

5.2 Tipi problemi di programmazione

Esistono problemi noti di programmazioni con soluzioni efficienti e avanzate. Questi problemi sono classificati in base alla forma della funzione obiettivo e dei vincoli.

5.2.1 Programmazione lineare

$$\begin{aligned} \min_x \quad & c^T x \\ \text{s.t.} \quad & Ax = b, \quad x \geq 0 \end{aligned}$$

Oppure, in maniera compatta:

$$\text{Minimizza } c^T x \text{ soggetto a } Ax = b \text{ e } x \geq 0$$

5.2.2 Programmazione quadratica

$$\begin{aligned} \min_x \quad & c^T x + \frac{1}{2} x^T Q x \\ \text{s.t.} \quad & Ax = b, \quad Cx \geq d \end{aligned}$$

Oppure, in maniera compatta:

$$\text{Minimizza } c^T x + \frac{1}{2} x^T Q x \text{ soggetto a } Ax = b \text{ e } Cx \geq d$$

Se la matrice Q è semi-definita positiva, allora il problema è convesso e ammette una soluzione unica. Una matrice Q è semi-definita positiva se:

$$x^T Q x \geq 0 \quad \forall x \in \mathbb{R}^n$$

5.2.3 Programmazione non lineare

$$\begin{aligned} \min_x \quad & f(x) \\ \text{s.t.} \quad & g(x) = 0, \quad h(x) \geq 0 \end{aligned}$$

Oppure, in maniera compatta:

$$\text{Minimizza } f(x) \text{ soggetto a } g(x) = 0 \text{ e } h(x) \geq 0$$

5.2.4 Problema duale - caso separabile

Per un problema quadratico è presente un problema duale ben noto:

$$\begin{aligned} \max_{\alpha} \quad & \sum_i \alpha_i - \frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j \\ \text{s.t.} \quad & \sum_i \alpha_i y_i = 0, \quad \alpha_i \geq 0, \quad \forall i \end{aligned}$$

Dato una soluzione $\alpha_1, \dots, \alpha_n$ al problema duale, la soluzione al problema primale è:

$$w = \sum_i \alpha_i y_i x_i$$

$$b = y_k - \sum_i \alpha_i y_i x_i^T x_k$$

Ogni α_i diverso da zero indica che la corrispondente x_i è un support vector. La funzione di classificazione è (nota che non serve calcolare esplicitamente w):

$$f(x) = \sum_i \alpha_i y_i x_i^T x + b$$

5.2.5 Problema duale - caso non separabile

Nel caso i punti non siano linearmente separabili, il problema duale diventa:

$$\begin{aligned} \max_{\alpha} \quad & \sum_i \alpha_i - \frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j \\ \text{s.t.} \quad & \sum_i \alpha_i y_i = 0, \quad 0 \leq \alpha_i \leq C, \quad \forall i \end{aligned}$$

di fatto cambiano le condizioni

5.2.6 Kernel trick

Se i dati non sono linearmente separabili nello spazio originale, possiamo provare a mappare i dati in uno spazio di dimensione superiore, dove potrebbero essere separabili. Il problema è che mappare i dati in uno spazio di dimensione superiore può essere computazionalmente costoso. Tuttavia, possiamo sfruttare il fatto che nel problema duale, i dati appaiono solo sotto forma di prodotti scalari $x_i^T x_j$. Supponendo di voler calcolare il prodotto scalare in uno spazio di dimensione superiore tramite una funzione di mappatura $\phi(x)$, possiamo definire una funzione kernel $K(x_i, x_j)$ che calcola direttamente il prodotto scalare nello spazio mappato.

$$K(x_i, x_j) = \phi(x_i)^T \phi(x_j)$$

E' dunque possibile trovare funzioni da applicare direttamente sui dati originali che simulano il calcolo del prodotto scalare nello spazio mappato, senza dover esplicitamente mappare i dati. Di fatto, facciamo un calcolo sapendo che è come se avessimo prima mappato i dati, ma saltando a tutti gli effetti la mappatura. Un esempio di kernel può essere:

$$K(x_i, x_j) = (1 + x_i^T x_j)^2$$

Teorema 1: Teorema di Mercer

- Ogni funzione simmetrica e semidefinita positiva è un kernel.
- Le funzioni simmetriche e semidefinite positive corrispondono a una matrice di Gram simmetrica e semidefinita positiva:

$$K = \begin{pmatrix} K(\mathbf{x}_1, \mathbf{x}_1) & K(\mathbf{x}_1, \mathbf{x}_2) & \cdots & K(\mathbf{x}_1, \mathbf{x}_n) \\ K(\mathbf{x}_2, \mathbf{x}_1) & K(\mathbf{x}_2, \mathbf{x}_2) & \cdots & K(\mathbf{x}_2, \mathbf{x}_n) \\ \vdots & \vdots & \ddots & \vdots \\ K(\mathbf{x}_n, \mathbf{x}_1) & K(\mathbf{x}_n, \mathbf{x}_2) & \cdots & K(\mathbf{x}_n, \mathbf{x}_n) \end{pmatrix}$$

Nb: una matrice simmetrica è semidefinita positiva se e solo se tutti gli autovalori sono non negativi.

Alcuni kernel comuni sono:

- Lineare:

$$K(x_i, x_j) = x_i^T x_j$$

- Polinomiale di grado p :

$$K(x_i, x_j) = (1 + x_i^T x_j)^p$$

- Gaussiana (funzione a base radiale):

$$K(x_i, x_j) = \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2}\right)$$

Mappatura $\Phi: x \rightarrow \varphi(x)$, dove $\varphi(x)$ è *infinito-dimensionale*.

5.3 Regularizzazione

Per controllare l'overfitting, si può aggiungere un termine di regularizzazione alla funzione di loss. In generale, l'obiettivo diventa:

$$\arg \min_{w, b} \sum_{i=1}^n \text{loss}(yy') + \lambda \text{regularizer}(w, b)$$

Un regularizer comune è la p -norma:

$$r(w, b) = \left(\sum_i |w_i|^p \right)^{1/p} = \|w\|^p$$

Definizione 13: Convessità

Matematicamente, una funzione $f: \mathbb{R}^n \rightarrow \mathbb{R}$ è detta *convessa* se per ogni coppia di punti $x_1, x_2 \in \mathbb{R}^n$ e per ogni $t \in (0, 1)$ vale che:

$$f(tx_1 + (1-t)x_2) \leq tf(x_1) + (1-t)f(x_2) \quad \forall 0 < t < 1$$

Teorema 2: *Convessità target con regolarizzazione*

Se la funzione di loss è convessa e il regularizzatore è convesso, allora la funzione obiettivo complessiva è convessa.

Le p -norms sono *convesse* per $p \geq 1$

6

Unsupervised learning - riduzione dimensione

L'idea generale è quella di "appirattire" i punti nell'asse parallela all'asse con varianza massima, così da ridurre al massimo la perdita di informazione

Formalmente si prende un versore w che parte da un punto c . Un versore significa un vettore con norma 1, ossia

$$|w| = 1 \Leftrightarrow w^T w = 1$$

dopodichè si condierano tutte le proiezioni dei punti x_i su questo versore, ossia

$$t_i = (x_i - c)^T w$$

6.0.1 Media e varianza

La media serve per il calcolo della varianza, la quale è il criterio secondo cui eseguiamo la riduzione di dimensione:

$$\begin{aligned} \mathbf{E}[t] &= \frac{1}{n} \sum_{i=1}^n t_i = \frac{1}{n} \sum_{i=1}^n (x_i - c)^T w \\ &= \bar{x}^T w - c^T w \end{aligned}$$

dove

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

La varianza indica quanto i valori sono sparsi l'ungo l'asse identificato dal versore w :

$$\begin{aligned}
\mathbf{Var}[t] &= \frac{1}{n} \sum_{i=1}^n (t_i - E[t])^2 \\
&= \frac{1}{n} \sum_{i=1}^n [x_i^\top w - c^\top w - \bar{x}^\top w + c^\top w]^2 \\
&= \frac{1}{n} \sum_{i=1}^n [(x_i - \bar{x})^\top w]^2 \\
&= \frac{1}{n} \sum_{i=1}^n (\bar{x}_i^\top w)^2 \\
&= \frac{1}{n} \sum_{i=1}^n w^\top \bar{x}_i \bar{x}_i^\top w \\
&= w^\top \left[\frac{1}{n} \sum_{i=1}^n \bar{x}_i \bar{x}_i^\top \right] w \\
&= w^\top \left[\frac{1}{n} \bar{X} \bar{X}^\top \right] w \\
&= w^\top C w
\end{aligned}$$

dove

$$t_i = (x_i - c)^\top w \quad \bar{X} = [\bar{x}_1, \dots, \bar{x}_n] \quad \bar{x}_i = (x_i - \bar{x})$$

e in particolare, C è della *matrice della covarianza*

Dunque abbiamo ottenuto che la varianza è data da:

$$\mathbf{Var}[t] = w^\top C w$$

dove C è la matrice della covarianza, definita come:

$$C = \begin{bmatrix} x_1 x_1 & \dots & x_1 x_n \\ x_2 x_1 & & x_2 x_n \\ \vdots & \ddots & \vdots \\ x_n x_1 & \dots & x_n x_n \end{bmatrix}$$

6.0.2 Autovalori e autovettori

Teorema 3: Condizioni soluzione a sistema

Dato un sistema lineare nella forma $Ax = 0$, allora questo presenta soluzioni diverse da 0 se e solo se

$$\det(A) = 0$$

Definizione 14: *Autovettore e autovalore*

Data una matrice A quadrata di dimensione $n \times n$ allora x si dice un suo autovettore relativo al valore λ se vale che

$$Ax = \lambda x$$

In particolare, per trovare autovettori e autovalori posso risolvere la seguente:

$$\det(A - \lambda I) = 0$$

questo perchè:

$$Av = \lambda v$$

$$Av - \lambda v = 0$$

$$(A - \lambda I)v = 0$$

la quale ha soluzioni non nulle se e solo se $\det(A - \lambda I) = 0$

Teorema 4: *Numero autovettori*

Data una matrice quadrata di dimensione $n \times n$, allora questa ha al massimo n autovettori linearmente indipendenti.

Teorema 5: *Numero autovettori in matrice simmetrica*

Data una matrice simmetrica quadrata di dimensione $n \times n$, allora questa ha esattamente n autovettori linearmente indipendenti.

6.0.2 Decomposizione di matrici

Data una matrice $A \in \mathbb{R}^{m \times m}$ simmetrica

- Allora esiste $U = [\mathbf{u}_1, \dots, \mathbf{u}_m] \in \mathbb{R}^{m \times m}$ and $\boldsymbol{\Lambda} = (\lambda_1, \dots, \lambda_m)^\top \in \mathbb{R}^m$ tale per cui:

$$A = U\Lambda U^\top$$

- $U \in \mathbb{R}^{n \times n}$ una matrice che ha per colonne gli *autovettori*
- $\Lambda \in \mathbb{R}^{n \times n}$ una matrice che ha sulla diagonale *gli autovalori* ordinati
- $U^\top U = UU^\top = I$

- O in alternativa

$$A = \sum_{j=1}^m \lambda_j \mathbf{u}_j \mathbf{u}_j^\top$$

- u_j autovettore
- λ_j il corrispondente autovalore
- Nota che gli *autovalori* vengono presi in ordine $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_m$

6.0.3 Obiettivo

L'obiettivo del nostro problema è trovare un sistema di riferimento in cui il primo asse sia parallelo alla direzione con varianza massima, formalmente:

$$w_i = \operatorname{argmax} \{w^\top C w : w^\top w = 1, \quad w \perp w_j \quad \forall j : 1 \leq j < i < m\}$$

si può dimostrare che la soluzione a questo problema di massimizzazione è dato dagli *autovettori* di C . In particolare, la i -esima componente principale è data dall' i -esimo autovettore di C .

Partiamo da

$$w_i^\top C w_i$$

la quale è scomponibile in per il teorema di decomposizione delle matrici in:

$$w_i^\top (U \Lambda U^\top) w_i$$

$$U^\top w_i = \begin{bmatrix} w_{11} & \dots & w_{1m} \\ \vdots & & \\ w_i & & \\ \vdots & & \\ w_{m1} & \dots & w_{mm} \end{bmatrix} \cdot \begin{bmatrix} w_{i1} \\ \vdots \\ w_{im} \end{bmatrix} = \begin{bmatrix} 0 \\ \vdots \\ \|w_i\|^2 \\ \vdots \\ 0 \end{bmatrix}$$

L'ultimo passaggio in quanto gli autovettori sono ortogonali tra loro, dunque il prodotto scalare tra autovettori diversi è 0.

$$\Lambda(U^\top w_i) = \begin{bmatrix} \lambda_1 & & \\ & \ddots & \\ & & \lambda_m \end{bmatrix} \cdot \begin{bmatrix} 0 \\ \vdots \\ \|w_i\|^2 \\ \vdots \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ \vdots \\ \lambda_i \|w_i\|^2 \\ \vdots \\ 0 \end{bmatrix}$$

$$U(\Lambda U^\top w_i) = \begin{bmatrix} w_{11} & \dots & w_{1m} \\ \vdots & & \\ w_{1i} & & \\ \vdots & & \\ w_{1m} & \dots & w_{mm} \end{bmatrix} \cdot \begin{bmatrix} 0 \\ \vdots \\ \lambda_i \|w_i\|^2 \\ \vdots \\ 0 \end{bmatrix} = \lambda_i \|w_i\|^2 \cdot \begin{bmatrix} w_{i1} \\ \vdots \\ w_{im} \end{bmatrix}$$

e infine

$$w_i^\top (U \Lambda U^\top w_i) = \lambda_i \|w_i\|^2 \cdot [w_{i1} \quad \dots \quad w_{im}] \cdot \begin{bmatrix} w_{i1} \\ \vdots \\ w_{im} \end{bmatrix} = \lambda_i \|w_i\|^2 \|w_i\|^2 = \lambda_i \|w_i\|^4$$

Da notare, di solito gli autovettori sono normalizzati, dunque $\|w_i\| = 1$, e quindi:

$$w_i^\top C w_i = \lambda_i$$

Per trovare gli autovettori di C ho due metodi:

Eigenvalue decomposition : data $A \in \mathbb{R}^{m \times m}$ *simmetrica* allora esiste

- $U = [u_1, \dots, u_m] \in \mathbb{R}^{m \times m}$
- $\lambda = [\lambda_1, \dots, \lambda_m]$

tale per cui

$$A = U\Lambda U^\top$$

dove

$$\Lambda = \begin{bmatrix} \lambda_1 & & \\ & \ddots & \\ & & \lambda_m \end{bmatrix}$$

SVN (Singular Value Decomposition) : data $A \in \mathbb{R}^{m \times n}$, allora esistono:

- $U = [u_1, \dots, u_m] \in \mathbb{R}^{m \times k}$
- $s = [s_1, \dots, s_n] \in \mathbb{R}^k$

tali per cui

$$A = USV^\top$$

dove

$$S = \begin{bmatrix} s_1 & & \\ & \ddots & \\ & & s_m \end{bmatrix}$$

6.0.4 Equivalenza metodi:

Dato il fatto che $C = \frac{1}{n}\bar{X}\bar{X}^\top$, allora posso

- Chiamare $U, \lambda = \text{eig}(C)$
- Chiamare $U, s, V = \text{SVN}(\bar{X})$ e calcolare

$$\lambda = \left[\frac{s^2}{n}, \dots, \frac{s^2}{n} \right]$$

L'equivalenza dei due metodi è data dal fatto che:

$$C = \frac{1}{n}\bar{X}\bar{X}^\top = \frac{1}{n}USV(USV)^\top = \frac{1}{n}(USV)(V^\top S^\top U^\top) = \frac{1}{n}US^2U^\top$$

quindi

$$C = \frac{1}{n}US^2U^\top$$

Algoritmo 10: *PCA with eigenvalue decomposition***Input:** Data points $X = [x_1, \dots, x_n]$ **Output:** Principal components $U = [u_1, \dots, u_m], \lambda = [\lambda_1, \dots, \lambda_m]$ **pca_eigen(X):**

▷ Centring points

$$\bar{X} = X - \frac{1}{n}X1_n1_n^\top;$$

▷ Compute covariance matrix

$$C = \frac{1}{n}\bar{X}\bar{X}^\top;$$

▷ Eigenvalue decomposition

$$U, \lambda = \text{eig}(C);$$

return U, λ ;dove 1_n è un vettore *colonna* di dimensione n con tutti gli elementi uguali a 1.**Algoritmo 11:** *PCA with SVD***Input:** Data points $X = [x_1, \dots, x_n]$ **Output:** Principal components $U = [u_1, \dots, u_k], \lambda = [\lambda_1, \dots, \lambda_n]$ **pca_svd(X):**

▷ Centring points

$$\bar{X} = X - \frac{1}{n}X1_n1_n^\top;$$

▷ SVN decomposition

$$U, s, V = \text{svd}(\bar{X});$$

▷ Computing λ

$$\lambda = \left(\frac{s_1^2}{n}, \dots, \frac{s_k^2}{n} \right);$$

return U, λ ;dove 1_n è un vettore *colonna* di dimensione n con tutti gli elementi uguali a 1.

Nota che per ridurre le dimensioni dei valori di un qualsiasi campione è sufficiente trovare la sua proiezione sulle componenti principali. Ricordando che la proiezione di b su a è data da $a \cdot b = a^\top b$, possiamo generalizzare al nostro caso.

Dato $\hat{W} = [w_1, \dots, w_k]$ contenente le componenti principali e $\bar{X} [x_1, \dots, x_n]$ contenente i campioni da proiettare, allora

$$T = \hat{W}^\top \bar{X}$$

T ha come colonne le proiezioni dei campioni sulle componenti principali

6.0.6 Trovare di quanto ridurre

Al fine di trovare un numero di dimensioni opportuno per cui si vuole effettuare la riduzione è possibile stimare la quantità di informazione persa con le seguenti formule:

$$\frac{\sum_{j=1}^k \lambda_j}{\sum_{j=1}^m C_{jj}}$$

Eigenvalue decomposition

$$\frac{\sum_{j=1}^k s_j^2}{\sum_{ij} \bar{X}_{ji}^2}$$

Singular value decomposition

queste due formule stimano la percentuale di informazione che viene ritenuta dopo aver ridotto la dimensione a k .

7 Clustering

7.1 k-means

- Dati dei punti $X = [x_1, \dots, x_n] \in \mathbb{R}^{d \times n}$
- Impostato un numero di *cluster* k
- Partiziona i punti in k cluster minimizzando la *variazione* totale

$$\min_{\mathcal{C}_1, \dots, \mathcal{C}_k} \sum_{j=1}^k V(\mathcal{C}_j)$$

- La variazione all'interno di un cluster $\mathcal{C}_j$ è definita come

$$V(\mathcal{C}_j) = \sum_{i \in \mathcal{C}_j} \|x_i - \mu_j\|^2$$

- Il centroide è solitamente dato dal "centro di massa" fra i punti:

$$\mu_j = \frac{1}{|\mathcal{C}_j|} \sum_{i \in \mathcal{C}_j} x_i$$

7.1.1 Pseudocodice

Algoritmo 12: *k-means*

```
k_clustering(X):  
  ▷ Initialization strategy to get initial cluster centroids  
     $\mu_1, \dots, \mu_k$   
  while clusters change do  
    for  $x_i \in X$  do  
      Assign  $x_i$  to the closest centroid, i.e.  
       $\mathcal{C}_j = \{i \in \{1, \dots, n\} : j = \arg \min_{\ell} \|x_i - \mu_{\ell}\|\}$   
      Compute new cluster centroids  $\mu_1, \dots, \mu_k$ ;  
  return  $\mathcal{C}_1, \dots, \mathcal{C}_k$ ;
```

7.1.2 Note

- Algoritmo termina sempre, ma potrebbe finire in *minimo locale*, non globale. La *loss function* non è convessa
- Risultati variano molto in base a come si scelgono i cluster iniziali. I metodi più comuni sono:
 - Random points (not examples) in the space
 - Randomly pick examples
 - Points least similar to any existing center (furthest centers heuristic)
 - Try out multiple starting points
 - Initialize with the results of another clustering method
- Complessità ad ogni iterazione:
 - $O(kn)$ per assegnare ognuno degli n punti ad un cluster k
 - $O(n)$ computazione centroidi
- Talvolta la distanza euclidea è poco adeguata, vedi [sezione 12.1](#)
- L'assunzione di base sulla distribuzione dei dati è che questi siano disposti in maniera "sferica" (generalizzabile a più dimensioni)

7.2 EM-clustering

L'EM assume che i dati provengano da una **miscela di distribuzioni gaussiane** (invece di cluster sferici come K-means). Ogni cluster è rappresentato da una gaussiana che può avere forma ellittica, permettendo di catturare cluster con forme più complesse.

- A differenza del K-means (assegnazione *hard*), qui un punto può appartenere a più cluster con probabilità diverse
- Ad esempio un punto può avere probabilità $p(\text{rosso}) = 0.8$ e $p(\text{blu}) = 0.2$

L'algoritmo alterna tra due passi principali:

- Per ogni punto x_i , calcola la *probabilità* $p(\theta_c|x_i)$ che appartenga a ciascun cluster c
- Ricalcola i parametri θ_c di ogni cluster (centro μ_c e matrice di covarianza Σ_c) usando le probabilità calcolate nel passo precedente
- Ogni punto contribuisce al nuovo centro in modo *pesato* secondo la sua probabilità di appartenenza

L'algoritmo continua ad alternare i due passi fino a convergenza:

$$\text{E-step: } p(\theta_c|x_i) = \frac{p(x_i|\theta_c)p(\theta_c)}{\sum_{j=1}^K p(x_i|\theta_j)p(\theta_j)} \quad (1)$$

$$\text{M-step: } \theta_c^{(\text{new})} = \arg \max_{\theta_c} \sum_{i=1}^N p(\theta_c|x_i) \log p(x_i|\theta_c) \quad (2)$$

L'algoritmo garantisce che la likelihood dei dati aumenti ad ogni iterazione (anche se può convergere a un ottimo locale).

Nota: $p(x_i | \theta_c)$ è la funzione gaussiana valutata in x_i . La generalizzazione della gaussiana in d dimensioni è:

$$\mathcal{N}[x; \mu, \Sigma] = \frac{1}{(2\pi)^{d/2} \sqrt{\det(\Sigma)}} \exp \left[-\frac{1}{2} (x - \mu)^T \Sigma^{-1} (x - \mu) \right]$$

dove Σ è la *matrice di covarianza* che descrive la forma dell'ellisse del cluster e μ è la *media*

7.3 Spectral clustering

Lo spectral clustering può aiutare nel momento in cui i dati non sono distribuiti in maniera gaussiana o circolare. L'algoritmo funziona così:

- Costruzione grafo secondo qualche criterio
- Partizione del grafo minimizzando il peso del taglio, ossia

$$\text{cut}(A, B) = \sum_{x \in A, y \in B} w(x, y)$$

più nello specifico, si cerca di minimizzare il taglio normalizzato:

$$\text{Ncut}(A, B) = \frac{\text{cut}(A, B)}{\text{vol}(A)} + \frac{\text{cut}(A, B)}{\text{vol}(B)}$$

nota che il taglio va *normalizzato* per evitare che il taglio sia influenzato dalla dimensione dei cluster, evitando tagli in cui ho 99 nodi da una parte e 1 dall'altra

Il problema può essere generalizzato ad un problema di autovalori

7.4 Clustering gerarchico

L'idea del clustering gerarchico è di procedere in maniera *agglomerativa* e *bottom-up*, aggregando gruppi simili

- Ogni elemento ξ è inizialmente un cluster
- Unisco gruppi secondo qualche criterio
- Termino quando rimane un singolo cluster

La struttura gerarchica ottenuta è detta *dendogramma*

Ciò che caratterizza un algoritmo è proprio come si definisce la distanza fra clusters. Ci sono molte opzioni quali: closest pair, farthest pair, average among all distances, etc

7.4.1 Pseudocodice

Formalmente, per risolvere il problema descritto in eq. (2), è necessario procedere come segue:

$$\begin{aligned}N_c &= \sum_{i=0}^k p(x \mid \theta_c) \\ \Sigma_c &= \frac{1}{N_c} + \sum_{i=1}^n p(\theta_c \mid x_i) \cdot \bar{x}_i \bar{x}_i^\top \\ \pi_c &= p(\theta_c) = \frac{N_c}{n}\end{aligned}$$

Algoritmo 13: *EM-clustering*

Input: Data points $X = [x_1, \dots, x_n] \in \mathbb{R}^{d \times n}$, number of clusters K

Output: K clusters with parameters $\theta_c = (\mu_c, \Sigma_c, \pi_c)$

em_clustering(X, K):

▷ Inizializzo π

for $c \in K$ **do**

└ $\pi_i = \frac{1}{K}$

while *clusters change* **do**

▷ E-step: calcolo le probabilità di appartenenza

for $c \in K$ **do**

$$\gamma_{ic} = p(\theta_c, \xi) = \frac{p(x_i | \theta_c) p(\theta_c)}{\sum_{j=1}^K p(x_i | \theta_j) p(\theta_j)};$$

▷ M-step: aggiorno gaussian parameters

for $c \in K$ **do**

$$N_c = \sum_{i=1}^n p(x_i | \theta_c);$$

$$\Sigma_c = \frac{1}{N_c} + \sum_{i=1}^n p(\theta_c | x_i) \cdot \bar{x}_i \bar{x}_i^\top;$$

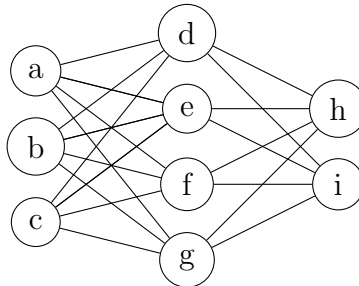
$$\pi_c = p(\theta_c) = \frac{N_c}{n};$$

7.4.2 Vantaggi rispetto a k-means

1. *Cluster ellittici*: Può gestire cluster con forme allungate, non solo circolari
2. *Clustering soft*: Fornisce probabilità di appartenenza invece di assegnazioni rigide
3. *Maggiore flessibilità*: La matrice di covarianza permette di catturare correlazioni tra le features

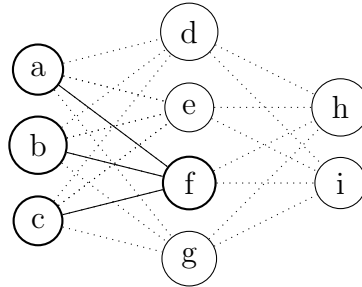
8

Reti neurali



8.1 Calcolo output predetto

Data una rete neurale con L layers, l'idea è che ogni nodo ha un'attivazione, ogni edge un peso. Per calcolare l'attivazione di un nodo t devo fare una "somma pesata" di tutti i nodi ai quali è collegato t .



in questo caso:

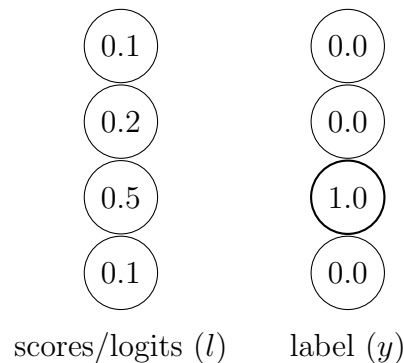
$$\text{Attivazione di } f = h(z_f = z_a + w_{af} + z_b + w_{bf} + z_c + w_{cf} + b_f)$$

dove b_f è il bias di f e h è la funzione di attivazione. In termini generali:

$$z_t = \sum_{i \in \mathcal{N}(t)} w_{it} z_i + b_t$$

8.1.1 Classificazione: softmax + cross-entropy

Una volta calcolate le attivazioni dell'ultimo layer per confrontarle con il risultato atteso (y) posso procedere come segue. Supponendo di avere



Softmax : la funzione *softmax* è definita come:

$$\text{softmax}(l) = S(l) = \frac{\exp(l_i)}{\sum_{j=1}^k \exp(l_j)}$$

intuitivamente:

- Applica esponenziale a ogni score z_i per enfatizzare gli outliers
- Normalizza il vettore l rendendolo una distribuzione di probabilità

questo equivale a creare il vettore:

$$S(l) = \left[\frac{e^{l_1}}{\sum_j e^{l_j}}, \dots, \frac{e^{l_m}}{\sum_j e^{l_j}} \right]$$

Cross-entropy loss : la *cross-entropy* è definita come:

$$\text{cross-entropy}(l, y) = \mathcal{L}(l, y) = \sum_k y_k \log(S(l)_k)$$

In termini vettoriali, dato l il vettore degli score e y ossia la label che indica il risultato atteso, allora la *cross-entropy* è definita come:

$$\mathcal{L}(l, y) = S(l)^\top y$$

Esempio : nel nostro caso:

$$S(l) = S([0.1, 0.2, 0.5, 0.1]) = \frac{1}{\sum_j e^j} [e^{0.1}, e^{0.2}, e^{0.5}, e^{0.1}]$$

calcolo la sommatoria:

$$\sum_i e^{l_i} = e^{0.1} + e^{0.2} + e^{0.5} + e^{0.1} \approx 3.2$$

dunque

$$S(l) = \frac{1}{3.2} [1.1, 1.2, 1.6, 1.1] \approx [0.34, 0.38, 0.5, 0.34]$$

8.1.2 Regression: softmax + cross-entropy

Se il vettore l degli scores deve dare origine ad una regressione di n valori allora posso calcolare la distanza dal valore atteso $\hat{y}$ con la loss quadratica:

$$\mathcal{L}(l, \hat{y}) = \frac{1}{2} \sum_{i=1}^n (l_i - \hat{y}_i)^2$$

nota che questa loss function è relativa *al singolo input* x_i

8.2 Backpropagation

L'idea della back propagation è di calcolare il gradiente della loss function rispetto ai pesi della rete neurale, in modo da poter aggiornare i pesi stessi tramite discesa del gradiente. L'algoritmo si basa sulla regola della catena per calcolare i gradienti in modo efficiente.

Notazione Inanzitutto prendiamo un sample x alla volta, con etichetta y in una rete con L livelli. La loss function è data da:

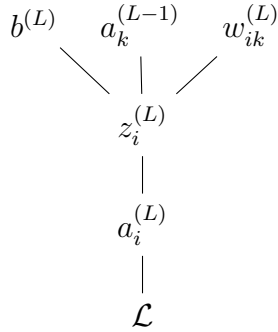
$$\mathcal{L} = \sum_j \left(a_j^{(L)} - y_j \right)^2$$

Indichiamo la *pre-attivazione* a livello L del neurone i con $z_i^{(L)}$. Il peso dal neurone i nel livello $L - 1$ al neurone j nel livello L è dato da $w_{ji}^{(L)}$ ("al contrario")

$$z_i^{(L)} = a_0^{(L-1)} w_{i0}^{(L)} + \dots + a_k^{(L-1)} w_{ik}^{(L)} = \sum_k a_k^{(L-1)} w_{ik}^{(L)}$$

dove k scorre tutti i neuroni del livello $L - 1$. L'attivazione del neurone i nel livello L è data da:

$$a_i^{(L)} = h\left(z_i^{(L)}\right)$$



Per la chain rule, il gradiente della loss function rispetto al peso $w_{ik}^{(L)}$ è dato da:

$$\frac{\partial \mathcal{L}}{\partial w_{ik}^{(L)}} = \frac{\partial \mathcal{L}}{\partial a_i^{(L)}} \cdot \frac{\partial a_i^{(L)}}{\partial z_i^{(L)}} \cdot \frac{\partial z_i^{(L)}}{\partial w_{ik}^{(L)}}$$

Derivata rispetto al peso e bias : in questo caso, siccome so che modificando il peso $w_{ij}^{(L)}$ andrò a modificare solo $z_i^{(L)}$, quindi mi basta considerare l'influenza di un singolo neurone nell'ultimo layer

$$\frac{\partial \mathcal{L}}{\partial w_{ik}^{(L)}} = \underbrace{\frac{\partial \mathcal{L}}{\partial a_i^{(L)}}}_1 \cdot \underbrace{\frac{\partial a_i^{(L)}}{\partial z_i^{(L)}}}_2 \cdot \underbrace{\frac{\partial z_i^{(L)}}{\partial w_{ik}^{(L)}}}_3$$

$$1 = 2 \left(a_i^{(L)} - y_i \right)$$

$$2 = h' \left(z_i^{(L)} \right)$$

$$3 = a_k^{(L-1)}$$

$$\mathcal{L} = \sum_j \left(a_j^{(L)} - y_j \right)^2$$

$$a_i^{(L)} = h \left(z_i^{(L)} \right)$$

$$z_i^{(L)} = \sum_k a_k^{(L-1)} w_{ik}^{(L)}$$

Derivata rispetto all'attivazione nel neurone precedente: a differenza di ciò che accade con bias e pesi, so che modificando l'attivazione $a_k^{(L-1)}$ tutti i neuroni nel layer L ne risentiranno. Per questo devo considerare la somma dei cambiamenti

$$\frac{\partial \mathcal{L}}{\partial a_k^{(L-1)}} = \sum_j \underbrace{\frac{\partial \mathcal{L}}{\partial a_j^{(L)}}}_1 \cdot \underbrace{\frac{\partial a_j^{(L)}}{\partial z_j^{(L)}}}_2 \cdot \underbrace{\frac{\partial z_j^{(L)}}{\partial a_k^{(L-1)}}}_3$$

$$1 = 2 \left(a_i^{(L)} - y_i \right)$$

$$2 = h' \left(z_i^{(L)} \right)$$

$$3 = w_{ik}$$

$$\mathcal{L} = \sum_j \left(a_j^{(L)} - y_j \right)^2$$

$$a_i^{(L)} = h \left(z_i^{(L)} \right)$$

$$z_i^{(L)} = \sum_k a_k^{(L-1)} w_{ik}$$

8.3 Reti neurali con layer specializzati

Le reti neurali possono essere arricchite con layer che eseguono operazioni diverse dalla somma pesata seguita da una funzione di attivazione. Le tre operazioni più comuni sono:

- Convoluzione → [sezione 8.3.1](#)
- Operazioni non lineari → [sezione 8.3.2](#)
- Operazioni di spatial pooling → [sezione 8.3.3](#)

8.3.1 Convoluzione

Una rete neurale che in almeno un layer effettua operazioni di convoluzione è detta *convolutional neural network* (CNN). Le CNN sono particolarmente efficaci per l'elaborazione di dati con struttura a griglia, come le immagini. La convoluzione in due dimensioni è così definita:

Definizione 15: Convoluzione discreta 2d

La convoluzione in due dimensioni è così definita:

$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(m, n) K(i - m, j - n)$$

Dove m e n vengono scelti in modo tale che $I(m, n)$ e $K(i - m, j - n)$ siano definiti.

Si noti che secondo questa definizione il kernel K viene "ribaltato" (*flipping*) prima di essere applicato all'immagine I .

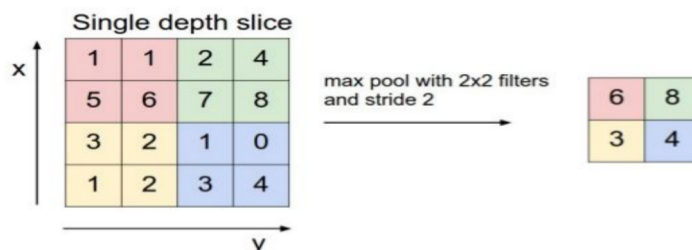
8.3.2 Operazioni non lineari

Una qualsiasi operazione non lineare può essere applicata dopo la convoluzione. Queste operazioni vengono applicate elemento per elemento. Ad esempio, in una griglia di pixel, posso applicare una operazioni non lineari ad ogni pixel individualmente, senza che questa abbia effetto sui suoi vicini. Un esempio di operazione non lineare può essere

$$h(x) = \max(0, x)$$

8.3.3 Operazioni di spatial pooling

Lo spatial pooling è una tecnica utilizzata nelle reti neurali convoluzionali per ridurre la dimensionalità spaziale delle rappresentazioni intermedie, mantenendo le informazioni più rilevanti. Le operazioni di pooling vengono applicate su piccole regioni dell'immagine o della mappa delle caratteristiche, aggregando i valori all'interno di queste regioni in un singolo valore rappresentativo. Un esempio classico è il *max-pooling* nelle immagini:



Il pooling è importante perché permette di ridurre la dimensionalità dei dati, diminuendo il numero di parametri e computazioni nella rete neurale. Questo oltre che ammortizzarne la complessità computazionale, aiuta a prevenire l'overfitting e rende la rete più robusta a piccole traslazioni o distorsioni nei dati di input.

8.4 Altri reti neurali

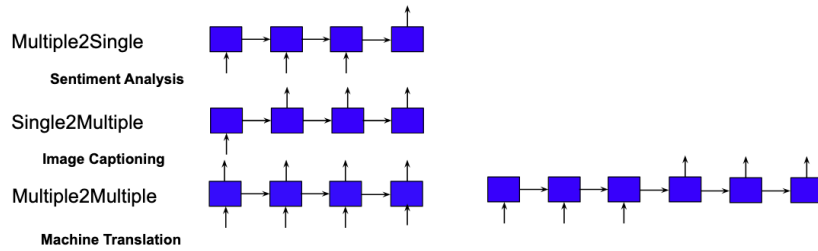
Esistono molte varianti di reti neurali progettate per compiti specifici. Alcune delle più comuni includono:

- Reti neurali ricorrenti (RNN) → [sezione 8.4.1](#)
- Autoencoders → [sezione 8.4.2](#)

8.4.1 Reti neurali ricorrenti (RNN)

Le reti neurali ricorrenti (RNN) sono progettate per elaborare dati sequenziali, come testo o serie temporali. A differenza delle reti neurali tradizionali, le RNN hanno connessioni cicliche che permettono di mantenere uno stato interno, consentendo loro di "ricordare" informazioni da input precedenti nella sequenza. Questo le rende particolarmente adatte per compiti come la traduzione automatica, il riconoscimento vocale e la generazione di testo o la predizione di fotogrammi di un video.

L'idea sta nel fatto di utilizzare l'output di un hidden layer come input per il successivo step temporale. In questo modo, l'informazione può essere mantenuta attraverso la sequenza, e fatta crescere con l'avanzare del tempo.

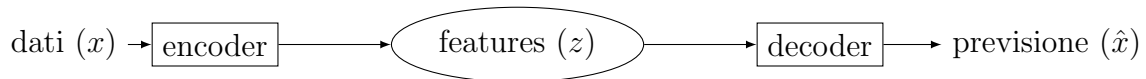


Più formalmente, ad ogni step temporale t , l'RNN aggiorna il suo stato nascosto h_t e produce un output y_t basato sull'input corrente x_t e sullo stato nascosto precedente h_{t-1} :

$$h_t = f_w(x_t, h_{t-1})$$

8.4.2 Autoencoders

Eseguono riduzione della dimensione



di fatto si ha che la loss function è data da:

$$\mathcal{L} = (\hat{x} - x)^2$$

dunque si allena una rete neurale per ridurre la dimensione dei dati in input e poi ricostruirli. In questo modo il layer centrale (features) conterrà una rappresentazione più compatta dei dati originali. Una volta allenata, si userà solo la prima parte (encoding), per produrre ad esempio features per un algoritmo di supervised learning

9 Modelli generativi

Ci sono due tipi principali di modelli generativi:

- Modelli generativi: cerco di indovinare la distribuzione dei dati p_X o p_{XY} a seconda del fatto che abbia o meno le etichette
- Modelli discriminativi: cerco la probabilità condizionata $p_{Y|X}$ per classificare i dati (l'output è una stima probabilistica della label data l'input)

I modelli discriminativi possono essere creati a partire da modelli generativi usando il teorema di Bayes:

$$p_{Y|X}(y | x) = \frac{p_{XY}(x, y)}{\sum_{y'} p_{XY}(x, y')}$$

9.1 Density estimation

Un problema di stima della densità è il seguente:

Definizione 16: *Density estimation - explicit*

Un problema di stima della densità esplicito consiste nel seguente:

Trova una distribuzione di probabilità $f \in \Delta(\mathcal{Z})$ che si adatti ai dati, dove $z \in \mathcal{Z}$ è campionato da una distribuzione dei dati sconosciuta $p_{\text{data}} \in \Delta(\mathcal{Z})$. dove

$$\begin{cases} \mathcal{Z} = X \times Y & \text{with labels (supervised learning)} \\ \mathcal{Z} = X & \text{without labels (unsupervised learning)} \end{cases}$$

Definizione 17: *Density estimation - implicit*

Un problema di stima della densità implicito consiste nel seguente:

Dato uno spazio Ω e uno spazio $\mathcal{Z}$, distribuiti rispettivamente secondo le distribuzioni $p_w \in \Delta(\Omega)$ e $p_{\text{data}} \in \Delta(\mathcal{Z})$, trovare una funzione $f \in \mathcal{Z}^\Omega$ in maniera tale che i campioni generati $f(z)$ siano distribuiti secondo p_{data}

$$\begin{cases} \mathcal{Z} = X \times Y & \text{with labels (supervised learning)} \\ \mathcal{Z} = X & \text{without labels (unsupervised learning)} \end{cases}$$

In entrambi i problemi l'obiettivo è quello di minimizzare la distanza fra la distribuzione dei dati e quella generata, ad esempio usando la distanza di Kullback-Leibler:

$$d_{KL}(p, q) = \mathbb{E}_{x \sim p} \left[\log \frac{p(x)}{q(x)} \right]$$

Il problema di minimizzazione è quindi dato da

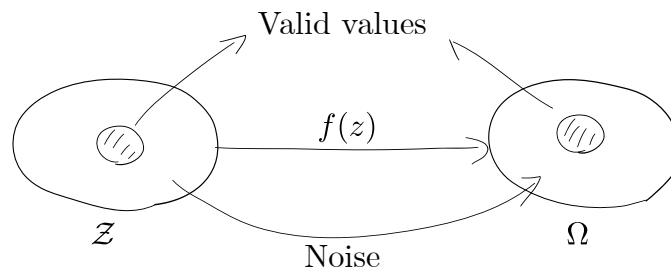
$$q^* \in \arg \min_{q \in \mathcal{H}} d(p_{\text{data}}, q)$$

9.2 VAE

Un autoencoder può effettuare dimensionality reduction come segue:

$$x \rightarrow \omega \rightarrow \hat{x}$$

l'obiettivo è quindi diminuire la dimensione di x e poi ricostruirla cercando di diminuire l'errore nella ricostruzione. Per generare nuovi dati potrei pensare di generare casualmente delle ω e poi decondificarle, ma questo non porterebbe a risultati utili, in quanto verrebbe prodotto rumore



L'idea è quindi di creare un decoder che ritorni una funzione di probabilità, non un dato certo. In questo modo possiamo esprimere la densità dello spazio reale p_{data} come una marginalizzazione della distribuzione latente p_{ω} :

$$q_{\theta}(x) = \mathbb{E}_{\omega \sim p_{\omega}} [q_{\theta}(x | \omega)]$$

dove la distribuzione $q_{\theta}(x | \omega)$ è l'output del decode e la distribuzione p^w è scelta a priori (*prior*, solitamente gaussiana).

Il problema di minimizzazione diventa dunque la ricerca di θ tale che la distanza della distribuzione generata sia minima rispetto a quella effettiva:

$$\theta^* \in \arg \min_{\theta \in \Theta} d(q_{\theta}, p_{\text{data}})$$

dove la distanza d è solitamente la distanza di Kullback-Leibler:

$$d_{KL}(p, q) = \mathbb{E}_{x \sim p} \left[\log \frac{p(x)}{q(x)} \right]$$

$$\begin{aligned} d_{KL}(p_{\text{data}}, q_{\theta}) &= \mathbb{E}_{x \sim p_{\text{data}}} \left[\log \frac{p_{\text{data}}(x)}{q_{\theta}(x)} \right] \\ &= -\mathbb{E}_{x \sim p_{\text{data}}} [\log q_{\theta}(x)] + \text{const} \\ &= -\mathbb{E}_{x \sim p_{\text{data}}} [\log \mathbb{E}_{\omega \sim p_{\omega}} [q_{\theta}(x | \omega)]] + \text{const} \end{aligned}$$

Sia $q_{\psi}(\omega | x) \in \Delta(\Omega)$ una distribuzione di probabilità, ossia quella generata da un encoder, che rappresenta una stima della distribuzione latente p_{ω} data l'osservazione x . Utilizzando l'*ineguaglianza di Jensen*, possiamo derivare un limite inferiore per il termine $\log \mathbb{E}_{\omega \sim p_{\omega}} [q_{\theta}(x | \omega)]$:

$$\begin{aligned} \log \mathbb{E}_{\omega \sim p_{\omega}} [q_{\theta}(x | \omega)] &= \log \mathbb{E}_{\omega \sim q_{\psi}(\cdot | x)} \left[q_{\theta}(x | \omega) \frac{p_{\omega}(\omega)}{q_{\psi}(\omega | x)} \right] \\ &\geq \mathbb{E}_{\omega \sim q_{\psi}(\cdot | x)} \left[\log \left(q_{\theta}(x | \omega) \frac{p_{\omega}(\omega)}{q_{\psi}(\omega | x)} \right) \right] \\ &= \mathbb{E}_{\omega \sim q_{\psi}(\cdot | x)} [\log q_{\theta}(x | \omega)] - d_{KL}(q_{\psi}(\cdot | x), p_{\omega}) \end{aligned}$$

Si noti che il primo passaggio è vero in quanto:

$$\mathbb{E}_p[f(\omega)] = \int f(\omega) p(\omega) d\omega = \int f(\omega) \frac{p(\omega)}{q(\omega)} q(\omega) d\omega = \mathbb{E}_q \left[f(\omega) \frac{p(\omega)}{q(\omega)} \right]$$

La formula finale da inserire dentro al problema di ottimizzazione è dunque:

$$d_{KL}(p_{\text{data}}, q_{\theta}) \leq \mathbb{E}_{x \sim p_{\text{data}}} [-\mathbb{E}_{\omega \sim q_{\psi}(\cdot | x)} [\log q_{\theta}(x | \omega)] + d_{KL}(q_{\psi}(\cdot | x), p_{\omega})] + \text{const}$$

dove

- $\mathbb{E}_{\omega \sim q_{\psi}(\cdot | x)} [\log q_{\theta}(x | \omega)]$ è detto *fattore di ricostruzione* in quanto promuove la generazione di immagini ricostruire simili a quelle reali in p_{data} . *Intractable*

- $d_{\text{KL}}(q_\psi(\cdot | x), p_\omega)$ è detto *fattore di regolarizzazione* in quanto spinge la distribuzione latente $q_\psi(\cdot | x)$ ad essere simile alla distribuzione a priori p_ω . Ha una *soluzione chiusa*

Dunque la funzione da minimizzare è data da:

$$\theta^* \in \arg \min_{\theta} \mathbb{E}_{x \sim p_{\text{data}}} \left[-\mathbb{E}_{\omega \sim q_\psi(\cdot | x)} [\log q_\theta(x | \omega)] + d_{\text{KL}}(q_\psi(\cdot | x), p_\omega) \right]$$

9.2.1 Conditional VAE

Se avessi delle labels, la formulazione matematica rimane simile, ma con la differenza che ora la distribuzione latente $q_\psi(\cdot | x)$ e la distribuzione generativa $q_\theta(x | \omega)$ dipendono anche dalla label y :

- *Distribuzione latente*: $q_\psi(\cdot | x, y)$
- *Distribuzione generativa* $q_\theta(x | w, y)$

9.2.2 Problemi

Anche se ora sono entrambi risolti, le VAE hanno due problemi principalmente:

- *Underfitting*: il regolarizzatore è troppo forte all'inizio dell'allenamento e spinge la distribuzione latente ad essere troppo simile alla distribuzione a priori, impedendo al modello di apprendere una rappresentazione utile dei dati
- *Blurry samples*: il generatore tende a produrre campioni sfocati, soprattutto quando la distribuzione dei dati è complessa e multimodale. Questo accade perché il modello cerca di mediare tra le diverse modalità della distribuzione, risultando in campioni che non rappresentano fedelmente nessuna delle modalità presenti nei dati reali.

9.3 GAN

I GAN permettono la stima implicita di densità. Supponiamo di avere una funzione di densità *a priori* $p_w \in \Delta(\Omega)$ e uno spazio di valori reali $\mathcal{X}$. Noi vogliamo trovare un mapping fra Ω e $\mathcal{X}$, ossia una funzione $g_\theta \in \Delta(\Omega)$ tale per cui la distribuzione indotta dai campioni generati da questa funzione sia il più simile possibile a quella di $\mathcal{X}$, ossia p_{data}

La distribuzione indotta da g_θ è data da:

$$q_\theta(x) = \mathbb{E}_{\omega \sim p^\omega} \delta(g_\theta(\omega) - x)$$

dove δ è la *delta di dirac*. Si noti come questa vada a selezionare solo i valori di ω che vengono mandati in x dalla funzione $g_\theta(\omega) - x$. Quindi intuitivamente, $q_\theta(x)$ è dato dalla media delle probabilità delle ω che vengono mandate in x dal generatore

Il problema risulta quindi quello di minimizzare la distanza fra q_θ e p_{data} :

$$\theta^* \in \arg \min_{\theta} d_{\text{JS}}(p_{\text{data}}, q_\theta)$$

sta volta, non useremo la *Kullback-Liebr* distance, ma la distanza di *Jensen-Shannon*, definita come:

$$\begin{aligned}
d_{JS}(p, q) &= \frac{1}{2} d_{KL}\left(p, \frac{p+q}{2}\right) + \frac{1}{2} d_{KL}\left(q, \frac{p+q}{2}\right) \\
&= \frac{1}{2} \mathbb{E}_{x \sim p} \left[\log \frac{2p(x)}{p(x) + q(x)} \right] + \frac{1}{2} \mathbb{E}_{x \sim q} \left[\log \frac{2q(x)}{p(x) + q(x)} \right] \\
&= \frac{1}{2} \mathbb{E}_{x \sim p} \left[\log \frac{p(x)}{p(x) + q(x)} \right] + \frac{1}{2} \mathbb{E}_{x \sim q} \left[\log \frac{q(x)}{p(x) + q(x)} \right] + \log(2) \\
&= \log(2) + \frac{1}{2} \max_t \{ \mathbb{E}_{x \sim p} [\log t(x)] + \mathbb{E}_{x \sim q} [\log(1 - t(x))] \}
\end{aligned}$$

Si può dimostrare che la funzione t che massimizza il secondo termine è data da:

$$t(x) = \frac{p(x)}{p(x) + q(x)}$$

per questo vale l'equivalenza fra le ultime due righe

$$d_{JS}(p_{\text{data}}, q_{\theta}) = \log(2) + \frac{1}{2} \max_t \{ \mathbb{E}_{x \sim p_{\text{data}}} [\log t(x)] + \mathbb{E}_{x \sim q_{\theta}} [\log(1 - t(x))] \}$$

Quindi il problema di ottimizzazione diventa:

$$\theta^* \in \arg \min_{\theta} \max_{\phi} \{ \mathbb{E}_{x \sim p_{\text{data}}} [\log t_{\phi}(x)] + \mathbb{E}_{x \sim q_{\theta}} [\log(1 - t_{\phi}(x))] \}$$

anche se dipendo ancora dalla esplicita funzione di densità q_{θ} , che non possono conoscere. Posso tuttavia applicare la funzione generatrice sugli elementi di Ω , ottenendo:

$$\theta^* \in \arg \min_{\theta} \max_{\phi} \{ \mathbb{E}_{x \sim p_{\text{data}}} [\log t_{\phi}(x)] + \mathbb{E}_{\omega \sim p^{\omega}} [\log(1 - t_{\phi}(g_{\theta}(\omega)))] \}$$

9.3.1 Problemi

- *Training stability*: i parametri potrebbero oscillare e non convergere mai
- *Mode collapse*: il generatore potrebbe imparare a generare perfettamente pochi esempi del training set
- *Vanishing gradient*: se il discriminatore è molto efficace, lascia al generatore poco gradiente da cui imparare

10 Diffusion models

L'idea alla base dei diffusion models è quella di imparare a generare dati partendo da rumore puro, invertendo un processo di diffusione che aggiunge rumore ai dati reali. In questo modo, il modello impara a ricostruire i dati originali a partire da una rappresentazione altamente rumorosa, consentendo la generazione di nuovi campioni che seguono la stessa distribuzione dei dati di addestramento.

Processo di diffusione : il processo di diffusione è un processo stocastico che aggiunge rumore ai dati reali in modo iterativo. A ogni passo del processo, viene aggiunta una piccola quantità di rumore gaussiano ai dati, rendendoli progressivamente più rumorosi. Matematicamente, il processo di diffusione può essere descritto come segue:

$$q(x_t | x_{t-1}) = \mathcal{N}(\sqrt{1 - \beta_t} x_{t-1}, \beta_t \mathbf{I}) \implies q(x_{1:T} | x_0) = \prod_{t=1}^T q(x_t | x_{t-1})$$

La formula si può anche scrivere con "accesso casuale", quindi senza che questa crei una catena di Markov, ma direttamente con x_0 e x_t . Definiamo innanzitutto il coefficiente $\bar{\alpha}_t$ ausiliario

$$\alpha_t = (1 - \beta_t) \implies \bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s)$$

Allora otteniamo che

$$q(x_t | x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t) \mathbf{I})$$

possiamo sfruttare la proprietà delle gaussiane per il campionamento. Data una gaussiana $z \sim \mathcal{N}(\mu, \sigma^2)$, allora posso campionare z come segue:

$$z = \mu + \sigma \cdot \epsilon, \quad \text{dove } \epsilon \sim \mathcal{N}(0, 1)$$

quindi per campionare x_t da $q(x_t | x_0)$, posso campionare $\epsilon \sim \mathcal{N}(0, 1)$ e poi calcolare

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \text{dove } \epsilon \sim \mathcal{N}(0, 1)$$

Processo di denoising : il processo di denoising è l'inverso del processo di diffusione. A ogni passo del processo di denoising, viene rimosso una piccola quantità di rumore dai dati rumorosi, avvicinandoli progressivamente ai dati originali. Matematicamente, il processo di denoising può essere descritto come segue:

$$p(x_T) \sim \mathcal{N}(0, \mathbf{I})$$

$$p_\theta(x_{t-1} | x_t) \sim \mathcal{N}\left(\underbrace{\mu_\theta(x_t, t)}_{\text{nn learn}}, \sigma_t^2 \mathbf{I}\right)$$

$$p_\theta(x_{0:T}) = p(x_T) \prod_{t=1}^T p_\theta(x_{t-1} | x_t)$$

Si noti che qui la probabilità è parametrizzata secondo i parametri θ di una rete neurale, che impara a stimare la media della distribuzione gaussiana che rappresenta il processo di denoising. Il processo di denoising viene addestrato minimizzando la distanza tra la distribuzione generata e quella dei dati reali, ad esempio usando la distanza di Kullback-Leibler:

Loss : è necessario come in qualsiasi algoritmo di machine learning definire una loss da minimizzare. In questo caso, la loss è data dalla distanza di Kullback-Leibler tra la distribuzione generata dal processo di denoising e quella dei dati reali:

$$L = \mathbb{E}_{x \sim q(x_0)} [-\log p_\theta(x)]$$

Tramite passaggi matematici, è possibile dimostrare che la loss da minimizzare è data da:

$$L_{\text{simple}} = \mathbb{E}_{x_0 \sim q(x_0), \epsilon \sim \mathcal{N}(0, \mathbf{I}), t \sim \mathcal{U}(1, T)} \left[\left\| \epsilon - \epsilon_\theta \left(\sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t \right) \right\|^2 \right]$$

10.1 Training

Algoritmo 14: *Training/inference diffusion models*

```
train():
    repeat
         $x_0 \sim q(x_0)$ ;
         $t \sim \text{Uniform}(\{1, \dots, T\})$ ;
         $\epsilon \sim \mathcal{N}(0, \mathbf{I})$ ;
        ▷ Aggiorna  $\theta$  con un passo di gradient descent
         $\nabla_{\theta} \|\epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon, t)\|^2$ ;
    until converged;

infer():
     $x_T \sim \mathcal{N}(0, \mathbf{I})$ ;
    for  $t = T$  to 1 do
         $z \sim \mathcal{N}(0, \mathbf{I})$ ;
         $x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(x_t, t) \right) + \sigma_t z$ ;
    return  $x_0$ ;
```

10.2 U-net

La U-NET è una particolare architettura di rete neurale convoluzionale progettata inizialmente per la segmentazione di immagini biomediche (ossia ad esempio analisi e classificazione tessuti, rilevamento tumori, ...). Ha due parti principali:

- *Encoder*: questa parte della rete è responsabile dell'estrazione delle caratteristiche dall'immagine di input. Consiste in una serie di strati convoluzionali e di pooling che riducono progressivamente la dimensione spaziale dell'immagine, aumentando al contempo la profondità dei canali di feature.
- *Decoder*: ripristina la risoluzione originale usando up-convolutions (o transposed convolutions) e combina le informazioni globali con dettagli locali (grazie ai "salti" laterali di connessione, skip connections).

Le caratteristiche principali della U-NET includono:

- *Fully convolutional*: non contiene strati completamente connessi, il che la rende adatta a lavorare con immagini di dimensioni variabili.
- *Skip connections*: permettono di mantenere dettagli fini durante la ricostruzione/-fase di upsampling.

I motivi principali per cui vengono usate come cuore nella fase di denoising dei diffusion models sono:

- Skip connections consentono di catturare sia informazioni globali che dettagli locali nell'immagine; aiutano a "recuperare" dettagli precisi persi nei passaggi di

compressione, cosa fondamentale per la qualità visiva delle immagini generate o denoise.

- Robusta, efficiente, e ampiamente studiata

10.3 Altre tecniche avanzate

Come ogni tecnologia, anche i diffusion models hanno subito numerose evoluzioni e miglioramenti. Di seguito sono riportate alcune delle tecniche avanzate più rilevanti

10.3.1 Diffusion GANs

I *Diffusion GANs* sono una variante dei modelli di diffusione (diffusion models) che combina i punti di forza dei GAN (Generative Adversarial Networks) e dei modelli di diffusione. Nel diffusion GAN, il processo di denoising non è più modellato solo come una gaussiana semplice (*unimodale*), ma tramite una distribuzione più complessa, appresa grazie a un GAN condizionale.

I diffusion models standard assumono che, a ogni step, il rumore venga rimosso con una gaussiana. Questa ipotesi va bene per passi molto piccoli (pochi cambiamenti), dunque servono migliaia di step per generare immagini realistiche. Per questo i diffusion GANs sono stati sviluppati per venire in contro alle seguenti esigenze:

- Ridurre il numero di passi necessari per la sintesi (generazione) dell'immagine
- Gestire tipi di rumore/denoising più complessi
- Accelerare la generazione senza sacrificare la qualità

Rispetto ad un GAN normale, ho i seguenti vantaggi:

- *Problema più semplice*: Sia il generatore che il discriminatore affrontano problemi meno difficili, perché operano su step locali nella catena di diffusione.
- *Stronger mode coverage*: La generazione copre meglio tutte le possibili forme e varianti dei dati: il GAN può apprendere anche distribuzioni multimodali.
- *Better training stability*: L'addestramento risulta più stabile rispetto a un GAN tradizionale, che tende a soffrire di "mode collapse" o instabilità.

10.3.2 Distillazione

Nel processo di denoising in una diffusion model, è necessario eseguire un numero elevato di passi (tipicamente migliaia) per ottenere risultati di alta qualità. Questo può essere computazionalmente costoso e lento. La distillazione è una tecnica che consente di ridurre il numero di passi necessari per la generazione, mantenendo comunque una buona qualità dell'immagine. L'idea è quella di addestrare un modello più piccolo (il "distillato" o student) a imitare il comportamento del modello originale (il "teacher") con meno passi, accelerando così il processo di generazione. Questa procedura può essere iterata (facendo diventare gli student i nuovi teachers)

10.3.3 Latent space diffusion models

L'idea è quella di combinare i VAE con i diffusion models per sfruttare i vantaggi di entrambi. In questo approccio, il VAE viene utilizzato per creare uno spazio latente compatto e strutturato, mentre il diffusion model viene applicato in questo spazio latente per generare nuove rappresentazioni.

- *Il VAE crea uno spazio latente*: L'encoder del VAE trasforma i dati (ad esempio immagini) in vettori compatti z_0 che costituiscono lo spazio latente
- *Il diffusion model genera rappresentazioni nel latente*: Invece di generare direttamente immagini, il modello di diffusione lavora nello spazio latente e produce nuove rappresentazioni $\omega \in \Omega$ (che sono punti/vettori in quello spazio)
- *Il VAE ricostruisce dal latente*: Il decoder del VAE prende queste rappresentazioni generate ed effettua la mappatura inversa, ricostruendo dati realistici (ad esempio immagini)

Vantaggi:

- *Latent embeddings semplici da modellare*: Lavorando in uno spazio latente (compresso, di dimensioni ridotte), sia il denoising che la sintesi diventano molto più efficienti rispetto a operare nello spazio originale ad alta dimensione (pixel per pixel).
- *Sintesi più veloce*: Generare in uno spazio più piccolo fa risparmiare tempo e memoria.
- *Augmented latent space*: Possibilità di progettare e arricchire lo spazio latente per ESIGENZE specifiche (ad esempio aggiungendo attributi, vettori di testo, etc.).
- *Autoencoder adattivi*: Puoi adattare l'autoencoder a diversi tipi di dati (immagini, testi, grafi, dati 3D...).

10.3.4 Text to image diffusion models

L'idea è di poter iniettare del testo per condizionare la generazione dell'immagine. In questo modo, è possibile generare immagini che corrispondono a descrizioni testuali specifiche, consentendo una maggiore personalizzazione e controllo sulla sintesi delle immagini.

Contrastive Language-Image Pretraining (CLIP) : è una tecnica per imparare uno spazio semantico condiviso tra immagini e testo. L'obiettivo è addestrare insieme (in modo "contrastivo") un text encoder e un image encoder in modo che rappresentazioni vettoriali (*embedding*) di testo e immagini "corrispondenti" siano vicine, mentre quelle non corrispondenti siano distanti nello spazio vettoriale.

Glide : è una tecnica che utilizza un modello di diffusione per generare immagini a partire da descrizioni testuali. Glide sfrutta un modello di diffusione condizionale, in cui il processo di denoising è guidato dalle rappresentazioni testuali ottenute da un encoder (ad esempio CLIP). In questo modo, il modello può generare immagini che corrispondono alle descrizioni fornite, consentendo una maggiore personalizzazione e controllo sulla sintesi

delle immagini. Formalmente, anziché avere $p_{\theta}(x_{t-1} | x_t)$ avrò $p_{\theta}(x_{t-1} | (x_t, y))$, dove y è la rappresentazione numerica del testo prompt

Stable diffusion : è una tecnica di latent space diffusion che utilizza un modello di diffusione per generare immagini a partire da descrizioni testuali, operando in uno spazio latente invece che direttamente nello spazio delle immagini. Di base viene creato uno spazio latente con le embedding di CLIP, e poi viene addestrato un modello di diffusione in questo spazio latente per generare nuove rappresentazioni. Infine, un decoder (ad esempio un VAE) viene utilizzato per ricostruire le immagini a partire dalle rappresentazioni generate nel latente. Questo approccio consente di ottenere una sintesi più veloce e di alta qualità, sfruttando i vantaggi sia dei modelli di diffusione che dei VAE.

11 Reinforcement learning

L'idea è quella che il modello, interagendo con l'ambiente può essere ricompensato oppure punito per delle scelte prese, e che quindi possa imparare a prendere decisioni migliori nel tempo. In questo modo, il modello può imparare a risolvere compiti complessi che richiedono una sequenza di azioni, come ad esempio giocare a un videogioco, controllare un robot, o prendere decisioni in un ambiente dinamico. In ogni scenario di reinforcement learning ho

- *State*: stato del sistem
- *Actions*: azioni che posso prendere per modificare lo stato
- *Reward*: ricompensa che ricevo in base alle azioni che prendo

L'obiettivo è quello di trovare una *policy* (ossia una strategia) che massimizzi la ricompensa totale nel tempo

11.1 Markov Decision Process (MDP)

$$(\mathcal{S}, \mathcal{A}, \mathcal{R}, \mathcal{P}, \gamma)$$

- $\mathcal{S}$: set of possible states
- $\mathcal{A}$: set of possible actions
- $\mathcal{R}$: distribution of reward given (state, action) pair
- $\mathcal{P}$: transition probability, i.e. distribution over next state given (state, action) pair
- γ : discount factor

Una catena di Markow può essere usata in un loop per continuare a prendere decisioni, come segue:

- Al tempo $t = 0$, l'ambiente campiona lo stato iniziale $s_0 \sim p(s_0)$
- Ripeti:
 - L'agente seleziona un'azione a_t

- L'ambiente campiona la ricompensa $r_t \sim R(\cdot \mid s_t, a_t)$
- L'ambiente campiona il prossimo stato $s_{t+1} \sim P(\cdot \mid s_t, a_t)$
- L'agente riceve la ricompensa r_t e il nuovo stato s_{t+1}

Una politica $\pi(s)$ è una funzione da $\mathcal{S}$ a $\mathcal{A}$ che specifica quale azione eseguire in ogni stato. L'obiettivo è trovare una politica π^* che massimizza la somma delle ricompense scontate cumulative.

Definizione 18: *Cumulative discounted reward*

La ricompensa totale scontata che un agente riceve seguendo una politica π a partire da uno stato s è data da:

$$\sum_{t \geq 0} \gamma^t r(s_t) \quad \text{dove } 0 < \gamma \leq 1$$

Si noti come il fattore di sconto γ riduce l'importanza delle ricompense future, rendendo più rilevanti le ricompense immediate. Inoltre si noti che:

- Il cumulative reward è *bounded* poiché la serie geometrica converge.
- Il fattore di sconto γ aiuta a garantire la convergenza dell'algoritmo di apprendimento, poiché riduce l'importanza delle ricompense future e rende il problema più gestibile.

Sono possibili due approcci principali per il Reinforcement Learning

- *Metodi basati sul valore*
 - L'obiettivo dell'agente è ottimizzare la funzione valore $V(s)$.
 - Il valore di ciascuno stato è la quantità totale di ricompensa che un agente RL si aspetta di raccogliere nel futuro a partire da uno stato dato.
- *Approccio basato sulla policy:*
 - Si definisce una policy che bisogna ottimizzare direttamente.
 - La policy determina il comportamento dell'agente.
 - Policy stocastica: fornisce una distribuzione di probabilità sulle diverse azioni.

$$\pi_\theta(s, a) \approx P(a \mid s)$$

11.2 Metodo basato sul valore

- La *funzione valore* fornisce la quantità totale di ricompensa che l'agente può aspettarsi di ottenere a partire da uno stato, sommando su tutti i possibili stati raggiungibili da quello stato. Grazie alla funzione valore si può trovare una policy.
- La *funzione valore* V di uno stato s rispetto alla policy π è la ricompensa cumulativa attesa seguendo quella policy a partire da s :

$$V^\pi(s) = \mathbb{E} \left[\sum_{t \geq 0} \gamma^t r(s_t) \mid s_0 = s, \pi \right]$$

dove $a_t = \pi(s_t)$ e $s_{t+1} \sim P(\cdot \mid s_t, a_t)$

- Il valore ottimale di uno stato è il valore ottenibile seguendo la migliore policy possibile:

$$V^*(s) = \max_{\pi} \mathbb{E} \left[\sum_{t \geq 0} \gamma^t r(s_t) \mid s_0 = s, \pi \right]$$

- Esprime quanto “buono” è uno stato.

Tuttavia si è visto che è opportuno calcolare il valore di una coppia stato-azione, invece che solo dello stato. Questo perché in alcuni stati, alcune azioni potrebbero essere migliori di altre, e quindi è importante valutare la qualità di ogni azione in ogni stato. La funzione che valuta la qualità di una coppia stato-azione è chiamata *funzione Q* (Q-value):

Definizione 19: Q-value function

La funzione Q-value indica il valore di una coppia stato-azione rispetto a una policy π . In altre parole, $Q^\pi(s, a)$ rappresenta la ricompensa cumulativa attesa che un agente riceverà seguendo la policy π a partire dallo stato s e prendendo l'azione a :

$$Q^\pi(s, a) = \mathbb{E} \left[\sum_{t \geq 0} \gamma^t r(s_t) \mid s_0 = s, a_0 = a, \pi \right]$$

- La funzione Q-value ottima indica quanto è buona una coppia stato-azione, assumendo che l'agente segua la migliore policy possibile dopo aver preso l'azione a nello stato s :

$$Q^*(s, a) = \max_{\pi} \mathbb{E} \left[\sum_{t \geq 0} \gamma^t r(s_t) \mid s_0 = s, a_0 = a, \pi \right]$$

- Dalla funzione Q-value ottima, è possibile derivare la policy ottima, che indica quale azione prendere in ogni stato per massimizzare la ricompensa cumulativa attesa:

$$\pi^*(s) = \arg \max_a Q^*(s, a)$$

$$\begin{aligned} Q^*(s, a) &= r(s) + \gamma \sum_{s'} P(s'|s, a) \max_{a'} Q^*(s', a') \\ &= \mathbb{E}_{s' \sim P(\cdot|s, a)} \left[r(s) + \gamma \max_{a'} Q^*(s', a') \mid s, a \right] \end{aligned}$$

Algoritmo 15: *Q-Learning training*

Data: State space $\mathcal{S}$, action space $\mathcal{A}$, reward function R , discount factor γ

Result: Optimal Q-values $Q^*(s, a)$

Initialize the Q matrix with zeros;

Select a random initial state s ;

for *each episode* **do**

while *state s is not goal state* **do**

 Select a random possible action a for the current state s ;

 Consider transition to next state s' using action a ;

 Get maximum Q value for s' : $\max_{a'} Q^*(s', a')$;

 Update $Q^*(s, a)$:

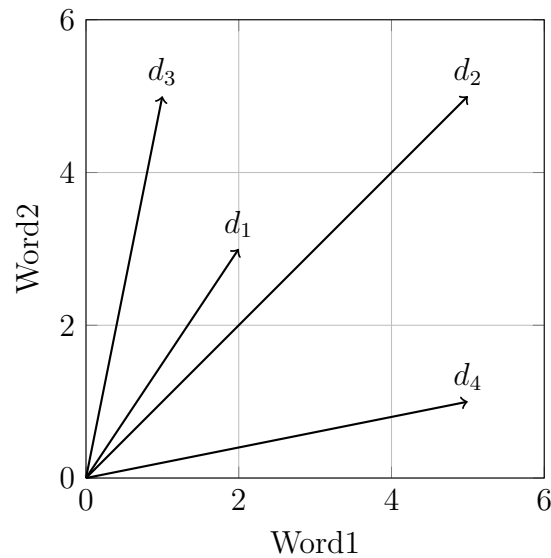
$Q^*(s, a) = R(s, a) + \gamma \max_{a'} [Q^*(s', a')]$;

 Set $s \leftarrow s'$;

12 Miscellanea nozioni

12.1 Cosine distance

In alcuni casi la distanza euclidea non è adeguata per valutare quanto due vettori sono simili. Ad esempio, se classificassimo documenti sulla base delle parole contenute e avessimo:



in questo caso d_1 e d_2 sono molto distanti mentre dal punto di vista logico risultano molto simili. Per questo è più efficiente la *cosine similarity*

$$\text{sim}(x, y) = \frac{x}{\|x\|} \cdot \frac{y}{\|y\|}$$

nota come $\text{sim}(x, y) \in [0, 1]$. Similmente:

$$d(x, y) = 1 - \text{sim}(x, y)$$